Compiladores (CC3001) — Resumo Teórico

Gonçalo Sousa Atualizado: Semana 3 (Aulas 1–6)

Conteúdo

Aula 1 — Introdução e fases de um compilador **2**
O que é um compilador . . . 2
Compiladores vs. interpretadores . . . 2
Fases de um compilador . . . 2
Self-hosting e bootstrapping . . . 3
Aula 2 — Análise lexical **4**
O que faz a análise lexical . . . 4
Expressões regulares . . . 4
Autómatos finitos determinísticos (DFA) . . . 5
Autómatos finitos não-determinísticos (NFA) . . . 6
Conversão de NFA em DFA: construção de subconjuntos . . . 7
Minimização de autómatos . . . 8
Aula 3 — Geradores de analisadores lexicais **10**
Implementação direta de um DFA em C . . . 10
Alex (gerador para Haskell) . . . 11
Flex (gerador para C) . . . 12
Longest match e first match . . . 12
Aula 4 — Análise sintática e gramáticas independentes de contexto **14**
O que é a análise sintática . . . 14
Gramáticas independentes de contexto . . . 14
Derivações . . . 14
Linguagem descrita por uma gramática . . . 15
Árvores sintáticas . . . 16
Gramáticas ambíguas . . . 17
Exemplo central: expressões aritméticas . . . 17
Eliminar ambiguidade: associatividade e precedência . . . 17
O problema do "dangling else" . . . 18
Aula 5 — Análise sintática *bottom-up* (LR) **20**
Análise *bottom-up* e analisadores LR . . . 20
Tabela de *parsing* LR e o algoritmo . . . 21
Autómato e tabela LR(0) . . . 23
Análise SLR(1) . . . 27
Conflitos e como resolvê-los . . . 29
Extras: LR(1) e LALR(1) . . . 30
Aula 6 — Geradores de analisadores sintáticos e sintaxe abstrata **31**
Geradores de analisadores LR . . . 31
Happy . . . 31
Ações semânticas . . . 33
Bison . . . 34
Conflitos, associatividade e precedência . . . 36
Sintaxe concreta e abstrata . . . 38
Representar sintaxe abstrata . . . 39

O sinal † marca matéria que não está nos slides mas faz falta para as práticas.

1 Aula 1
Introdução e fases de um compilador

O que é um compilador

Um **compilador** traduz um programa de uma **linguagem fonte** para um programa equivalente numa **linguagem destino**, normalmente de alto nível (C, Java) para baixo nível (código máquina, *bytecode* JVM). Exemplos: GCC (C/C++ → máquina), Javac (Java → JVM), GHC (Haskell → máquina), Scalac (Scala → JVM). O destino também pode ser de alto nível: o `elm` compila Elm para JavaScript.

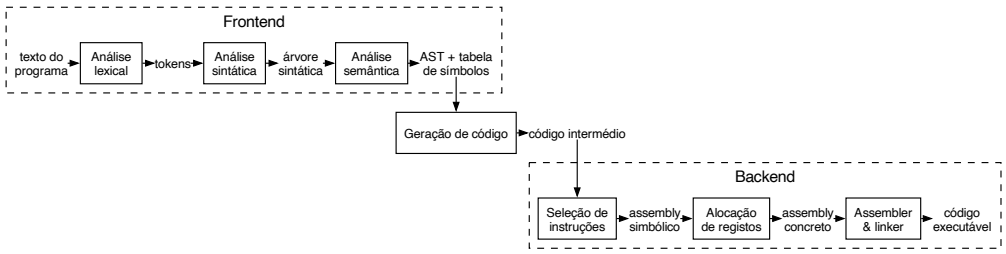
Compiladores vs. interpretadores

> **Definição** **Compilação vs. interpretação**
>
> **Compilação**: traduz o programa inteiro **antes** de o executar, e produz um executável reutilizável. **Interpretação**: faz a mesma análise lexical e sintática, mas **executa diretamente** o programa (a partir da árvore sintática ou de código intermédio), sem produzir executável.

	Vantagem
Interpretação	Mais simples de implementar; suporta várias arquiteturas com facilidade; permite REPL (*read-eval-print-loop*).
Compilação	Código normalmente mais eficiente; o executável distribui-se sem o compilador.

Muitas implementações combinam as duas (GHC compila e interpreta; OCaml tem `ocaml` e `ocamlopt`).

Fases de um compilador



> **Definição** **Frontend / Backend**
>
> **Frontend** (depende da linguagem fonte, não da máquina): análise lexical → análise sintática → análise semântica → AST + tabela de símbolos.
> **Backend** (depende da máquina, não da linguagem): seleção de instruções → alocação de registos → *assembler* & *linker*.
> Entre os dois, o **código intermédio**, que não depende de nenhuma das duas.

A separação simplifica o compilador e permite **reutilizar componentes**: o *backend* do GCC serve C, C++ e Objective-C; o LLVM é partilhado por Clang, Swift, Rust e GHC. Este semestre o foco é o **frontend**.

Self-hosting e bootstrapping

Definição **Self-hosting e bootstrapping**

Um compilador é **self-hosted** se está escrito na linguagem que compila (GCC em C, Javac em Java, GHC em Haskell, Rustc em Rust).

A primeira versão faz-se por **bootstrapping**: escreve-se um compilador noutra linguagem, usa-se para compilar uma versão escrita na própria linguagem, e repete-se até o compilador se compilar a si próprio (ex.: o primeiro compilador de Rust foi escrito em OCaml).

2 Aula 2 Análise lexical

O que faz a análise lexical

O **léxico** de uma linguagem é o conjunto dos seus *tokens*: **identificadores**, **literais** (números, carateres, cadeias), **palavras reservadas** (`if`, `while`, …) e **operadores**. O **analisador lexical** (*lexer*, *scanner*) parte o texto numa sequência de *tokens*, para o *parser* trabalhar sobre *tokens* já classificados e não sobre carateres.

Definição **Representação de um token**

Cada *token* tem uma **etiqueta** e, nalguns casos, um **valor**:

Entrada	Etiqueta	Valor
`if`	`IF`	—
`,`	`COMMA`	—
`!=`	`NOTEQ`	—
`main`	`ID`	`"main"`
`71`	`NUM`	`71`
`.5`	`REAL`	`0.5`

Exemplo resolvido

`float match0(char *s) { if (!strncmp(s, "0.0", 3)) return 0.0; }` dá:
`FLOAT, ID("match0"), LPAREN, CHAR, STAR, ID("s"), RPAREN, LBRACE, IF, LPAREN, BANG, ID("strncmp"), LPAREN, ID("s"), COMMA, STRING("0.0"), COMMA, NUM(3), RPAREN, RPAREN, RETURN, REAL(0.0), SEMI, RBRACE, EOF`
Espaços, mudanças de linha e comentários **não geram tokens**: só separam *tokens*. O `EOF` marca o fim da entrada.

Expressões regulares

Escrever o *lexer* à mão (um `switch` gigante) é repetitivo e fácil de errar. A alternativa é **descrever os tokens com expressões regulares** e gerar o analisador automaticamente (Aula 3).

Definição **Expressões regulares (REs)**

Dado um alfabeto Σ, uma **linguagem** é um conjunto $L \subseteq \Sigma^*$. As expressões regulares são:

- a — uma letra $a \in \Sigma$ (reconhece só `"a"`);
- ε — a palavra vazia;
- $M \mid N$ — **união**: o que M ou N reconhecem;
- $M \cdot N$ — **concatenação**: $u \cdot v$ com $u \in L(M)$ e $v \in L(N)$;
- M^* — **repetição** (fecho de Kleene): zero ou mais palavras de M.

A concatenação escreve-se implícita ($abc = (a \cdot b) \cdot c$) e a prioridade é repetição > concatenação > união: $ab|c = (ab)|c$ e $abb^* = ab(b^*)$.

Abreviatura	Significado
`M?`	opcional: $M \mid \varepsilon$
`M+`	uma ou mais vezes: $M \cdot M^*$
`[x1x2...xn]`	um dos carateres: $x_1 \| x_2 \| \ldots \| x_n$
`[a-z]`	intervalo de carateres
`[^a-z]`	qualquer carácter fora do intervalo
`.`	qualquer carácter exceto mudança de linha
`"a+"`	cadeia literal (o + não é operador)

Exemplo resolvido

Os *tokens* de um pequeno analisador em C:

```
if                                  palavra reservada (IF)
[_a-zA-Z][_a-zA-Z0-9]*              identificadores (ID)
[0-9]+                              número inteiro (NUM)
([0-9]+"."[0-9]*)|([0-9]*"."[0-9]+) número fracionário (REAL)
//.*                                comentário até ao fim da linha
```

`[_a-zA-Z][_a-zA-Z0-9]*` também reconhece `if`: a ordem das regras resolve isto (*first match*, Aula 3).

Parar no primeiro terminador †. Para um comentário `/* ... */`, a expressão `"/*" .* "*/"` não serve: `.*` consome o máximo possível, e em `/* a */ x /* b */` reconhece tudo como **um** comentário, do primeiro `/*` ao último `*/`. A técnica é proibir o terminador no meio. Por exemplo, para `<!-- ... -->`: `"<!--" ([^-]|"-"[^-])* "-->"` (qualquer carácter que não seja –, ou um – que não é seguido de outro –). Para `*/` a ideia é a mesma.

Pratica agora **Expressões regulares (Folha lab. 2, Exercício 2)**

Papel e lápis, usando só esta secção:

- **(a)** — expressões regulares para todos os tokens do `scanner-c` (`ID`, `NUM`, `LPAREN`, `RPAREN`, `COMMA`, `IF` e os do Exercício 1: chavetas, `;`, `WHILE`, `FOR`, `INT`, `FLOAT`, `REAL`). Não te esqueças do _ nos identificadores.
- **(b)** — generaliza `REAL` para notação científica (`12.34e+12`, `1e-12`, `123.4E+9`): repara que em `1e-12` não há ponto.
- **(c)** — comentários `// ...` e `/* ... */`. Para o multi-linha usa a técnica "parar no primeiro terminador" acima, adaptada a `*/`.

Soluções: Ex. 2 (a), Ex. 2 (b), Ex. 2 (c)

Autómatos finitos determinísticos (DFA)

As expressões regulares dizem **o que** reconhecer; para implementar o analisador é preciso um modelo que diga **como**: um autómato finito.

Definição **DFA**

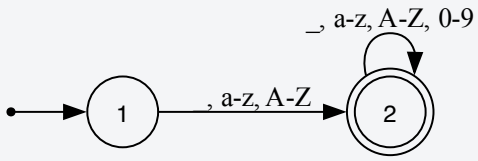
Um **autómato finito determinístico** é $(\Sigma, Q, q_0, F, \delta)$:

- Σ — alfabeto; Q — conjunto finito de estados;
- $q_0 \in Q$ — estado inicial; $F \subseteq Q$ — estados finais;
- $\delta \subseteq Q \times \Sigma \times Q$ — transições, com **no máximo um** q' para cada q e σ (ou seja, uma função parcial $q' = \delta(q, \sigma)$).

Uma palavra é **aceite** se, a partir de q_0 e seguindo as transições dos seus símbolos, se termina num estado de F.

Exemplo resolvido

DFA de `[_a-zA-Z][_a-zA-Z0-9]*` (identificadores):



$Q = \{1, 2\}$, $q_0 = 1$, $F = \{2\}$; δ tem $(1, \ell, 2)$ para cada letra ou _ ℓ, e $(2, \ell, 2)$, $(2, d, 2)$ para cada letra ou _ ℓ e dígito d. Para x1: $1 \xrightarrow{x} 2 \xrightarrow{1} 2$, e $2 \in F$: **aceite**.

Autómatos finitos não-determinísticos (NFA)

Definição **NFA**

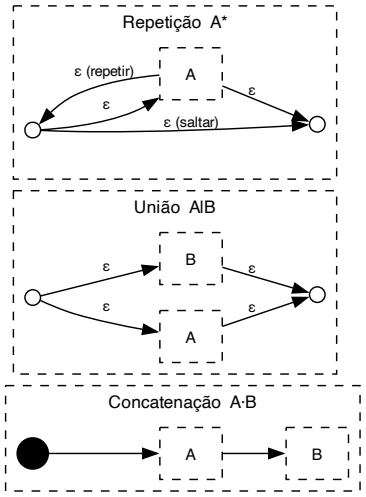
Igual a um DFA, mas com $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$:

- pode haver **várias transições** com o mesmo símbolo a partir do mesmo estado;
- há **transições-ε**, que mudam de estado **sem consumir** nenhum símbolo.

Um NFA não se implementa diretamente (pode estar em vários estados ao mesmo tempo). Por isso o caminho é **expressão regular → NFA (fácil) → DFA equivalente (implementável)**.

Construção de Thompson: de expressão regular a NFA

Cada sub-expressão vira um **fragmento** de autómato com uma entrada e uma saída, e os fragmentos combinam-se pela estrutura da expressão:



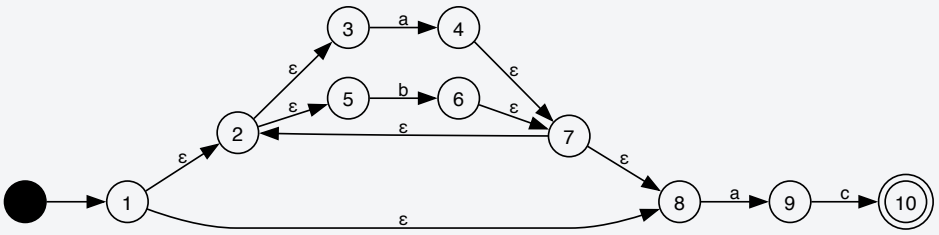
Definição **Regras de Thompson**

- **Símbolo** σ (ou ε): uma transição σ da entrada para a saída.
- **Concatenação** $A \cdot B$: a saída de A liga à entrada de B.
- **União** $A \mid B$: entrada nova com ε para as entradas de A e B; as saídas de A e B ligam por ε a uma saída nova.
- **Repetição** A^*: entrada e saída novas; a entrada liga por ε à entrada de A e diretamente à saída (zero repetições); a saída de A liga por ε à entrada de A (repetir) e à saída nova (terminar).

Exemplo resolvido

NFA de $(a \mid b)^* \cdot a \cdot c$ (o exemplo da aula):

1. Fragmento `a` (estados 3→4) e fragmento `b` (5→6).
2. **União** `a|b`: estado 2 (entrada, ε para 3 e 5) e estado 7 (saída, ε a partir de 4 e 6).
3. **Repetição** `(a|b)`$*$: par 1 (entrada) / 8 (saída); 1 → 2 (entrar), 1 → 8 (zero repetições), 7 → 2 (repetir), 7 → 8 (terminar).
4. **Concatenação** com `a`: $8 \xrightarrow{a} 9$.
5. **Concatenação** com `c`: $9 \xrightarrow{c} 10$ (final).



Conversão de NFA em DFA: construção de subconjuntos

Definição **closure e construção de subconjuntos**

Para um NFA $\mathcal{A} = (\Sigma, Q, q_0, F, \delta)$:

$$\text{closure}(S) = \{\, q' \in Q : \text{há } q \in S \text{ e um caminho só de transições-}\varepsilon \text{ de } q \text{ a } q' \,\}$$

(os estados alcançáveis a partir de S **sem consumir símbolos**). O DFA equivalente $(\Sigma, 2^Q, S_0, F', \delta')$ tem como estados **conjuntos de estados do NFA**:

$$S_0 = \text{closure}(\{q_0\}) \qquad F' = \{\, S : S \cap F \neq \varnothing \,\} \qquad \delta'(S, \sigma) = \text{closure}(\{\, q' : q \in S,\ (q, \sigma, q') \in \delta \,\})$$

Exemplo resolvido

Construção de subconjuntos para o NFA de $(a \mid b)^*ac$ acima. Transições-ε: 1→2, 1→8, 2→3, 2→5, 4→7, 6→7, 7→2, 7→8. Com símbolo: $3 \xrightarrow{a} 4$, $5 \xrightarrow{b} 6$, $8 \xrightarrow{a} 9$, $9 \xrightarrow{c} 10$.

Estado inicial: $S_0 = \text{closure}(\{1\}) = \{1, 2, 3, 5, 8\}$ (de 1: $1 \to 2$, $2 \to 3$, $2 \to 5$, $1 \to 8$; nenhum destes tem mais ε).

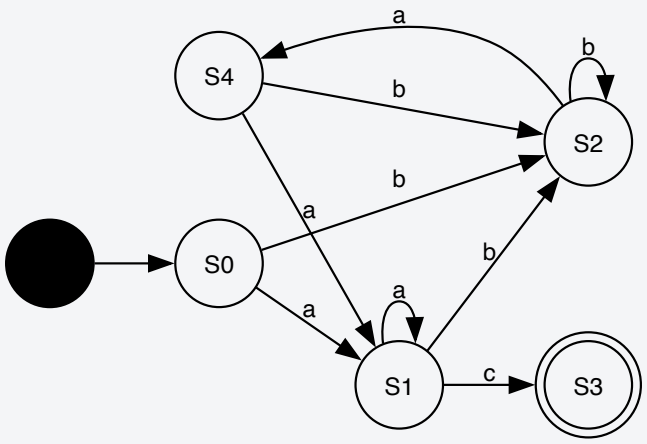
- S_0 **com a**: em S_0 só 3 e 8 têm a, move $= \{4, 9\}$. closure($\{4, 9\}$): $4 \to 7$, $7 \to 2$, $7 \to 8$, $2 \to 3$, $2 \to 5$; o 9 não tem ε. Dá $\{2, 3, 4, 5, 7, 8, 9\}$, novo: S_1.
- S_0 **com b**: só 5 tem b, move $= \{6\}$. closure($\{6\}$): $6 \to 7$, $7 \to 2$, $7 \to 8$, $2 \to 3$, $2 \to 5$. Dá $\{2, 3, 5, 6, 7, 8\}$, novo: S_2.
- S_0 **com c**: só o 9 tem c, e $9 \notin S_0$: sem transição.

Exemplo resolvido

Continuação, com $S_1 = \{2, 3, 4, 5, 7, 8, 9\}$ e $S_2 = \{2, 3, 5, 6, 7, 8\}$:

- S_1 **com a**: 3 e $8 \in S_1$, move $= \{4, 9\}$, closure $= S_1$: ciclo em a.
- S_1 **com b**: $5 \in S_1$, move $= \{6\}$, closure $= S_2$.
- S_1 **com c**: $9 \in S_1$, move $= \{10\}$, closure($\{10\}$) $= \{10\}$, novo: S_3. Contém o final 10, logo $S_3 \in F'$.
- S_2 **com a**: só $3 \in S_2$ (o 8 não está em S_2), move $= \{4\}$. closure($\{4\}$): $4 \to 7$, $7 \to 2$, $7 \to 8$, $2 \to 3$, $2 \to 5$. Dá $\{2, 3, 4, 5, 7, 8\}$, novo: S_4.
- S_2 **com b**: $5 \in S_2$, move $= \{6\}$, closure $= S_2$: ciclo em b.
- S_2 **com c**: nenhum estado de S_2 tem c: sem transição.
- S_4 **com a**: 3 e $8 \in S_4$, move $= \{4, 9\}$, closure $= S_1$.
- S_4 **com b**: $5 \in S_4$, move $= \{6\}$, closure $= S_2$.
- S_4 **com c**: sem transição.

Não aparecem estados novos: o DFA tem S_0 a S_4, com $F' = \{S_3\}$.



Minimização de autómatos

O DFA dos subconjuntos pode ter **estados equivalentes**: as mesmas transições para todo o alfabeto e o mesmo estatuto de aceitação. Fundindo-os obtém-se o **autómato mínimo**. Os geradores (Alex, Flex) minimizam automaticamente.

Reconhecer estados equivalentes é uma pergunta típica de exame. No exemplo acima, $S_0 = \{1, 2, 3, 5, 8\}$ e $S_4 = \{2, 3, 4, 5, 7, 8\}$ têm as mesmas transições ($\xrightarrow{a} S_1$, $\xrightarrow{b} S_2$, nada com c) e nenhum é final. Os estados em que diferem

(1 de um lado, 4 e 7 do outro) não têm transições que consumam símbolos. Logo $S_0 \equiv S_4$, e o autómato mínimo tem 4 estados.

3 Aula 3
Geradores de analisadores lexicais

Esta aula cobre o último passo de **expressão regular → NFA → DFA → implementação**: implementar um DFA, e **gerar** esse código automaticamente a partir das expressões regulares.

Implementação direta de um DFA em C

Um DFA implementa-se com uma **tabela de transições** `delta[estado][símbolo]` e um ciclo que lê carateres:

Exemplo resolvido

DFA dos identificadores, com o estado `0` a representar "erro" e o estado final `2`:

```c
#define STATES  3    // estados 0 (erro), 1, 2
#define SYMBOLS 128  // códigos ASCII
const int delta[STATES][SYMBOLS] = {
  /* estado 0 */ { 0, ..., 0, ... },            // erro é absorvente
  /* estado 1 */ { 0, ..., 2, ..., 2, ... 0 },  // letra/_ -> 2
  /* estado 2 */ { 0, ..., 2, ..., 2, ... 0 } };// letra/_/dígito -> 2

int state = 1;         // estado inicial
int c;
while ((c = getchar()) != EOF) {
  state = delta[state][c];
  if (state == 0) return ERROR;    // rejeita
  if (FINAL(state)) return ACCEPT; // aceita (FINAL(state) é state==2)
}
return ERROR; // fim do input sem atingir estado final
```

Traço com a entrada `"x1"`:

Passo	`c` lido	`delta[state][c]`	novo `state`	ação
1	`x` (letra)	`delta[1]['x']` = 2	2	`FINAL(2)` → **`return ACCEPT`** já

O ciclo **nunca lê o `1`**: aceita no primeiro estado final que atinge, não implementa *longest match*. Um *lexer* real guarda a posição do último estado final visto e continua a ler até não haver transição (ver "Longest match e first match", abaixo). Cada novo *token* exigiria mais tabelas escritas à mão: é esse trabalho que os geradores automatizam.

Pratica agora **DFA em C (Folha lab. 2, Exercício 1)**

O `src/CLexer.c` do `scanner-c` segue o padrão "ler carácter, decidir por casos" desta secção (um `switch/while`, sem tabela explícita). Estende-o:

- **(a)**, **(b)** — novos tokens `{`, `}`, `;` e palavras reservadas `WHILE`, `FOR`, `INT`, `FLOAT`: o mesmo padrão de `IF/LPAREN` já lá.
- **(c)** — aceitar `_` como letra: é a expressão regular `[_a-zA-Z]` mal transcrita para o C (falta um `case '_':` a par de `isalpha(c)`).
- **(d)** — `REAL` (`0.5`, `123.45`): um estado novo depois do `.`.
- **(e)** — ignorar comentários `/* ... */` e `// ...`.
- **(f)** — *buffer overrun* nos identificadores: testa o comprimento máximo antes de escrever no buffer.

Cuidado com a ordem `if` vs. `ID` (ver "Longest match e first match").
Soluções: Ex. 1 (a), Ex. 1 (b), Ex. 1 (c), Ex. 1 (d), Ex. 1 (e), Ex. 1 (f)

Alex (gerador para Haskell)

Definição **Alex**

Gerador de analisadores lexicais para Haskell. Recebe um ficheiro (`Lexer.x`) com regras "expressão regular → ação em Haskell" e produz `Lexer.hs`: `Lexer.x` $\xrightarrow{\texttt{alex}}$ `Lexer.hs` $\xrightarrow{\texttt{ghc}}$ executável.

Exemplo resolvido

```
{
module Lexer where
}
%wrapper "basic"
$alpha = [_a-zA-Z]
$digit = [0-9]

tokens :-
$white+                     ;  -- ignorar carateres brancos
if                          { \_ -> IF }
$alpha($alpha|$digit)*      { \s -> ID s }
$digit+                     { \s -> NUM (read s) }
$digit+"."$digit+           { \s -> REAL (read s) }

{
data Token = IF | ID String | NUM Int | REAL Double
}
```

O que está entre `{ }` é Haskell; o resto são diretivas, definições e regras `padrão { ação }`, em que a ação é uma função `String -> Token`. O `Lexer.hs` exporta `alexScanTokens :: String -> [Token]`:

```
module Main where
import Lexer
main = do
  txt <- getContents
  print (alexScanTokens txt)
```

Com `%wrapper "posn"` cada *token* recebe também a sua **posição**: a ação passa a ter tipo `AlexPosn -> String -> Token`, com `data AlexPosn = AlexPn Int Int Int` (posição absoluta, linha, coluna), ex.: `\pos s -> ID pos s`.

Flex (gerador para C)

Definição **Flex**

O equivalente do Alex para C. Recebe um `.l` e produz `lex.yy.c`: `ficheiro.l` $\xrightarrow{\texttt{flex}}$ `lex.yy.c` $\xrightarrow{\texttt{gcc}}$ executável.

Exemplo resolvido

```
%{
#include "tokens.h"
%}
%option noyywrap
alpha       [_a-zA-Z]
digit       [0-9]
%%

[ \t\n\r]+                /* skip whitespace */
if                         { return IF; }
{alpha}({alpha}|{digit})*  { return ID; }
{digit}+                   { yynum = atoi(yytext); return NUM; }
{digit}+"."{digit}+        { yyreal = atof(yytext); return REAL; }
<<EOF>>                    { return EOF; /* end of input */ }
```

Estrutura: `%{ declarações %}`, definições, `%%`, regras `padrão { ação em C }`, e opcionalmente outro `%%` com código C.

```c
#include "tokens.h"
int main(void) {
  int token;
  while ((token = yylex()) != EOF) {
    switch (token) {
      case IF:   printf("IF ");               break;
      case ID:   printf("ID(%s) ", yytext);    break;
      case NUM:  printf("NUM(%d) ", yynum);    break;
      case REAL: printf("REAL(%f) ", yyreal);  break;
    }
  }
}
```

`yylex()` devolve o próximo *token* (um inteiro, definido num *header*). A variável global `yytext` tem o texto do *token* atual; os valores de `NUM`/`REAL` guardam-se em globais (`yynum`, `yyreal`).

O Flex não guarda posições: com `%option yylineno` mantém o número da linha em `yylineno`, mas é o programador que a guarda em cada *token* †.

Longest match e first match

Definição **Regras de desambiguação**

Quando várias regras casam com a entrada, o Alex e o Flex escolhem, por esta ordem:

1. **Longest match**: a regra que consome **mais carateres**;
2. **First match**: em caso de empate, a regra que aparece **primeiro** no ficheiro.

Consequência (pergunta típica): `if` casa com a regra `if` e com a de identificadores, ambas com 2 carateres. Por *first match*, as **palavras reservadas têm de vir antes dos identificadores**; com a ordem trocada, `if` sairia sempre como `ID`. E `iffy` sai como `ID` por *longest match*.

Pratica agora **Flex (Folha lab. 2, Exercício 3)**

- Reimplementa o `scanner-c` num ficheiro `Lexer.l`, com a estrutura de três partes da secção "Flex" acima e as expressões regulares que escreveste no Exercício 2. Compila com `flex Lexer.l` e `gcc lex.yy.c -o Lexer`.
- Põe as palavras reservadas **antes** da regra de `ID` (*first match*) e confirma que `iffy` sai como `ID` (*longest match*).

Soluções: Ex. 3

	Alex	Flex
Linguagem gerada	Haskell (`.hs`)	C (`lex.yy.c`)
Ficheiro de entrada	`.x`	`.l`
Texto do token	argumento da ação (`\s -> ...`)	global `yytext`
Posição do token	`%wrapper "posn"` (automático)	`%option yylineno` (manual)
Arranque	`alexScanTokens :: String -> [Token]`	`yylex()` em ciclo

4 Aula 4
Análise sintática e gramáticas independentes de contexto

O que é a análise sintática

A **análise sintática** verifica se a sequência de *tokens* respeita a estrutura da linguagem (parêntesis e chavetas casados, operadores com o número certo de operandos, instruções separadas por **;**, …) e constrói a **árvore sintática**. Não verifica se o programa faz sentido: isso é a **análise semântica**, a fase seguinte. A estrutura especifica-se com uma **gramática independente de contexto**.

Gramáticas independentes de contexto

Definição **Gramática independente de contexto (CFG)**

Um quádruplo $G = (\Sigma, N, S, P)$:

- Σ — **terminais** (os *tokens* do analisador lexical);
- N — **não-terminais** (categorias sintáticas: "expressão", "instrução", …);
- $S \in N$ — símbolo **inicial**;
- P — **produções** $X \to \alpha$, com $X \in N$ e α uma sequência (possivelmente vazia) de terminais e não-terminais.

"Independente de contexto": o lado esquerdo é sempre **um só não-terminal**, por isso uma produção aplica-se sem olhar ao que está à volta.

Gramática usada ao longo desta aula (a dos slides), com as produções numeradas: $\Sigma = \{a, b\}$, $N = \{S, B\}$, inicial S.

Nº	Produção
(1)	$S \to aSB$
(2)	$S \to \varepsilon$
(3)	$S \to B$
(4)	$B \to Bb$
(5)	$B \to b$

Derivações

Definição **Derivação**

$\Rightarrow$ substitui um não-terminal pelo lado direito de uma das suas produções. $\Rightarrow^*$ é zero ou mais passos. Quando há vários não-terminais, escolhe-se qual substituir †:

- **mais à esquerda** (*leftmost*): sempre o mais à esquerda;
- **mais à direita** (*rightmost*): sempre o mais à direita.

As duas dão a **mesma árvore**. A análise *top-down* (LL) constrói uma derivação mais à esquerda; a *bottom-up* (LR, Aula 5) constrói uma derivação mais à direita, de trás para a frente.

Exemplo resolvido

Derivação **mais à esquerda** de $aabbb$ (número da produção por cima):

$$S \overset{1}{\Rightarrow} aSB \overset{1}{\Rightarrow} aaSBB \overset{2}{\Rightarrow} aaBB \overset{4}{\Rightarrow} aaBbB \overset{5}{\Rightarrow} aabbB \overset{5}{\Rightarrow} aabbb$$

A produção 1 aplicada duas vezes dá dois `a` à esquerda e dois B à direita; a 2 apaga o S; a 4 transforma o primeiro B em Bb; a 5 resolve esse B em `b`, e outra vez a 5 resolve o último B.
Derivação **mais à direita** da mesma árvore (sublinhado: o próximo a substituir):

$$S \overset{1}{\Rightarrow} aS\underline{B} \overset{5}{\Rightarrow} a\underline{S}b \overset{1}{\Rightarrow} aaS\underline{B}b \overset{4}{\Rightarrow} aaS\underline{B}bb \overset{5}{\Rightarrow} aa\underline{S}bbb \overset{2}{\Rightarrow} aabbb$$

Os dois B resolvem-se antes do S do meio, porque estão à direita dele.

Linguagem descrita por uma gramática

Definição **L(G)**

$L(G) = \{\, w \in \Sigma^* \,:\, S \Rightarrow^* w \,\}$: as palavras só de terminais que se derivam do símbolo inicial.

Exemplo resolvido **Exercício 1(a) (Folha lab. 3, também nos slides)**

Que linguagem descreve a gramática acima? Onde ficam os `a` e os `b`, e qual a relação entre as quantidades?

- B gera **um ou mais `b`**: $B \to b$ dá `b`, e $B \to Bb$ junta mais um. Logo B gera b^+ (não há $B \to \varepsilon$).
- $S \to aSB$ aplicada n vezes dá $a^n\, S\, B_1 \cdots B_n$, e cada B_i dá b^{k_i} com $k_i \geq 1$.
- O S do meio acaba de uma de duas formas:
  - (2) $S \to \varepsilon$: o total de `b` é $m = k_1 + \cdots + k_n \geq n$, e qualquer $m \geq n$ se atinge (aumentando um dos k_i). Com $n = 0$ dá a palavra vazia.
  - (3) $S \to B$: junta mais um B, ou seja $m \geq n + 1$. Estas palavras **já se obtinham** pela via (2): a produção (3) não acrescenta palavras, só **derivações alternativas** (é o que torna a gramática ambígua, ver abaixo).

$$L(G) = \{\, a^n b^m \,:\, n \geq 0,\ m \geq n \,\}$$

Numa frase: **zero ou mais `a` seguidos de zero ou mais `b`, com pelo menos tantos `b` como `a`**. Os `a` estão sempre todos antes dos `b`.

Escrever uma gramática para uma linguagem

Não há algoritmo geral, mas uma **expressão regular** traduz-se mecanicamente †, com um não-terminal novo para cada $*$, `+` ou `|`:

Expressão regular	Produções
rs (concatenação)	um lado direito com r seguido de s
$r \mid s$ (alternativa)	$X \to r$ e $X \to s$
r^* (zero ou mais)	$X \to r\,X \mid \varepsilon$
r^+ (uma ou mais)	$X \to r\,X \mid r$

Por isso **toda a linguagem regular é independente de contexto**. O contrário é falso: $\{a^n b^m \,:\, m \geq n\}$ não é regular (um autómato finito não consegue contar os `a`).

Exemplo resolvido **Exercício 1(d) (Folha lab. 3)**

Gramática para `((ab*a)|(ba*b))`.

1. **Alternativa de topo**: `ab*a` ou `ba*b`, duas produções para S.
2. **`ab*a`**: `b*` precisa de um não-terminal, $B \to bB \mid \varepsilon$; a concatenação fica $S \to aBa$.
3. **`ba*b`**: da mesma forma, $A \to aA \mid \varepsilon$ e $S \to bAb$.

$$S \to aBa \mid bAb \qquad B \to bB \mid \varepsilon \qquad A \to aA \mid \varepsilon$$

Verificação: `abba` é aceite, $S \Rightarrow aBa \Rightarrow abBa \Rightarrow abbBa \Rightarrow abba$. `ab` não é: começa por `a`, só pode vir de $S \to aBa$, que acaba em `a`.

Pratica agora **Escrever gramáticas (Folha lab. 3, Exercício 1)**

Ex. 1(b) Gramática para as palavras com o mesmo número de `a`s e `b`s, por qualquer ordem. Aqui não há expressão regular (a linguagem não é regular). Pista: olha para a **primeira** letra da palavra e para o ponto onde as contagens voltam a ficar iguais pela primeira vez.
Ex. 1(c) Gramática para os parêntesis casados (`()`, `(())`, `()()`, …). Pista: uma palavra não vazia começa por um `(` que fecha num certo `)`; o que está lá dentro e o que vem a seguir são outra vez parêntesis casados.
Ex. 1(e) Gramática para `((0|1)+"."(0|1)*)|((0|1)*"."(0|1)+)`. Usa a tabela acima, como no exemplo da 1(d). Um não-terminal para "um dígito" (`0|1`) simplifica tudo.
A 1(a) e a 1(d) estão resolvidas acima.
Soluções: Ex. 1 (b), Ex. 1 (c), Ex. 1 (e)

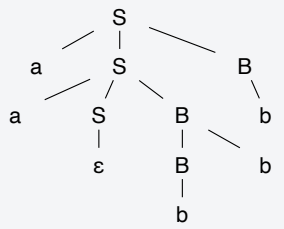
Árvores sintáticas

Definição **Árvore sintática**

Uma produção $X \to \alpha_1 \dots \alpha_n$ usada numa derivação é um nó X com n filhos, um por cada símbolo α_i, da esquerda para a direita. Esta árvore, com um nó por símbolo, é a **árvore sintática concreta**; o compilador guarda uma versão compacta, a **árvore sintática abstrata** (Aula 6).

Exemplo resolvido

Árvore da derivação de *aabbb* acima:



A raiz S usa (1): filhos `a`, o S do meio e um B (o "exterior"). O S do meio usa (1): `a`, o S interno e um segundo B. O S interno usa (2) e fica vazio. O segundo B usa (4), com filhos B (que usa (5)) e `b`. O B exterior usa (5). Folhas, da esquerda para a direita: `a`, `a`, `b`, `b`, `b`.

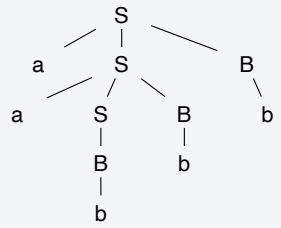
Gramáticas ambíguas

Definição **Ambiguidade**

Uma gramática é **ambígua** se alguma palavra tem **duas ou mais árvores sintáticas diferentes**. Duas derivações diferentes não chegam: podem dar a mesma árvore (só muda a ordem dos passos).

Exemplo resolvido

A gramática acima é ambígua. `aabbb` também se obtém com (3) $S \to B$ no S interno, em vez de (2): $S \overset{1}{\Rightarrow} aSB \overset{1}{\Rightarrow} aaSBB \overset{3}{\Rightarrow} aaBBB \overset{5}{\Rightarrow} aabBB \overset{5}{\Rightarrow} aabbB \overset{5}{\Rightarrow} aabbb$. Ficam três B, todos resolvidos por (5), sem usar (4):



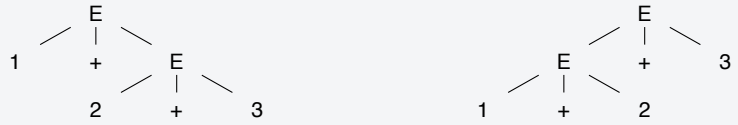
A árvore é diferente da anterior e a palavra é a mesma: a gramática é ambígua.

Porque importa. Para *descrever* uma linguagem, uma gramática ambígua serve. Mas o compilador usa a árvore para dar **significado** ao programa: com duas árvores, não sabe qual escolher. Por isso as gramáticas das linguagens de programação não são ambíguas, ou têm regras explícitas de desambiguação. Em exame, para mostrar que uma gramática é ambígua, exibem-se **duas árvores** diferentes para a mesma palavra.

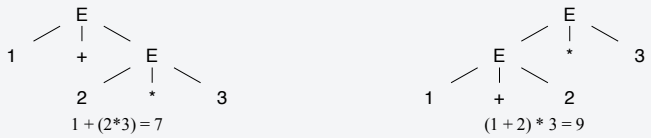
Exemplo central: expressões aritméticas

Exemplo resolvido

$E \to E{+}E \mid E{*}E \mid$ `num` $\mid (E)$ é ambígua. `1+2+3` tem duas árvores (mesmo resultado, 6, mas árvores diferentes):



E em `1+2*3` as duas árvores dão **resultados diferentes**:



Conforme a árvore construída, `1+2*3` valeria 7 ou 9.

Eliminar ambiguidade: associatividade e precedência

Muitas vezes elimina-se a ambiguidade **reescrevendo a gramática**, sem mudar a linguagem. Há duas coisas a fixar:

- **associatividade**: `1+2+3` é ((1+2)+3) ou (1+(2+3))?

- **precedência**: `1+2*3` é $1 + (2 \times 3)$ ou $(1+2) \times 3$?

A alternativa, sem mexer na gramática, é declarar associatividade e precedência no gerador de analisadores (Aula 6).

Definição **Gramática de expressões desambiguada**

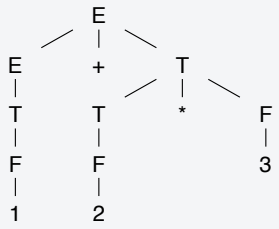
$$E \to E + T \mid T \qquad T \to T * F \mid F \qquad F \to \texttt{num} \mid (E)$$

Uma **expressão** (E) é uma soma de **termos** (T), um termo é um produto de **fatores** (F), e um fator é um número ou uma expressão entre parêntesis.

- A **recursão à esquerda** ($E \to E + T$, não $E \to T + E$) dá **associatividade à esquerda**.
- Como um $*$ só aparece **dentro** de um T, agarra os seus operandos antes de a soma os usar: **$*$ tem mais precedência que `+`**.

Exemplo resolvido

Com esta gramática, `1+2*3` só tem uma árvore:



A raiz usa $E \to E + T$: à esquerda $E \to T \to F \to 1$; à direita um T que usa $T \to T * F$, com $T \to F \to 2$ e $F \to 3$. O $*$ fica preso no ramo direito do `+`: o resultado é sempre $1 + (2 \times 3) = 7$.

Pratica agora **Ambiguidade em programas sequenciais (Folha lab. 3, Exercício 2)**

Gramática de "programas sequenciais" (também um exercício dos slides): $S \to S ; S \mid \texttt{ident} = E \mid \texttt{ident++}$ e $E \to \texttt{ident} \mid \texttt{num} \mid E + E$.

Ex. 2(a) Mostra que é ambígua, com duas árvores diferentes para a mesma frase. O enunciado pergunta se há mais do que um exemplo: há, em **dois** sítios distintos. Um é nas expressões (`E→E+E`, o mesmo problema do `E→E+E` acima). O outro é nas próprias instruções: `S→S;S` tem a mesma forma que `E→E+E`, por isso tem o mesmo problema de associatividade.

Ex. 2(b) Reescreve a gramática sem ambiguidade. Usa a recursão à esquerda de `E→E+T|T` acima, aplicada nos dois sítios (também a `S`).

Soluções: Ex. 2 (a), Ex. 2 (b)

O problema do "dangling else"

$$S \to \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S \mid \texttt{if}\ E\ \texttt{then}\ S \mid \ldots$$

Com o `else` opcional, `if e1 then if e2 then s1 else s2` tem duas árvores: o `else` liga-se ao `if e2` (o mais próximo) ou ao `if e1`. A convenção de quase todas as linguagens é **o `else` liga-se ao `if` mais próximo**. Impõe-se de duas formas:

- **Corrigir a gramática**, com M (*matched*: todo o `if` tem `else`) e U (*unmatched*: há um `if` por fechar):

$$S \to M \mid U \qquad M \to \texttt{if}\ E\ \texttt{then}\ M\ \texttt{else}\ M \mid \ldots \qquad U \to \texttt{if}\ E\ \texttt{then}\ S \mid \texttt{if}\ E\ \texttt{then}\ M\ \texttt{else}\ U$$

  Um `if–then–else` só é M se os dois ramos forem M, o que obriga o `else` a agarrar o `if` mais próximo.

- **Deixar a gramática** e resolver no analisador: um analisador LR escolhe *shift* no conflito (Aula 5, "Conflitos e como resolvê-los"). É o mais comum.

5 Aula 5
Análise sintática *bottom-up* (LR)

Como se constrói um analisador que aceita ou rejeita uma sequência de *tokens* e descobre a árvore: o funcionamento de um analisador LR (pilha + tabela), a tabela **LR(0)**, a melhoria **SLR(1)**, os **conflitos** e, como extra, **LR(1)** e **LALR(1)**.

Análise *bottom-up* e analisadores LR

Definição **Análise LR**

LR = *Left-to-right parse, Rightmost derivation*: lê a entrada da esquerda para a direita e constrói uma **derivação mais à direita** ao contrário, das folhas (os *tokens*) até ao símbolo inicial. Por isso é ***bottom-up***; a análise LL (*top-down*) parte do símbolo inicial.

Em relação à LL, a análise LR reconhece **mais linguagens**, é **mais fácil reescrever** uma gramática para ela, e permite **resolver ambiguidades** com **prioridades** e **associatividades**, sem alterar a gramática (Aula 6).

Definição **Analisador LR (autómato de pilha)**

Tem uma **entrada** (os terminais por ler) e uma **pilha** de símbolos (terminais e não-terminais), no início vazia. Em cada passo faz:

- ***Shift***: tira o próximo terminal da entrada e empilha-o.
- ***Reduce***: para uma produção $X \to \gamma$ com γ no **topo** da pilha, tira γ e empilha X.

Acrescenta-se à gramática um símbolo inicial S', o terminal \$ (**fim da entrada**) e a produção $S' \to S\,\$$.

Exemplo resolvido **Exemplo 1 dos slides: parêntesis equilibrados**

Gramática: $S' \to S\,\$$ e $S \to (S)S \mid \varepsilon$. Entrada `()`.

Pilha	Entrada	Ação	Porquê
ε	`()$`	shift	nada no topo para reduzir
`(`	`)$`	reduce $S \to \varepsilon$	dentro dos parêntesis há um S (vazio)
`(S`	`)$`	shift	
`(S)`	`$`	reduce $S \to \varepsilon$	falta o S final de $(S)S$; é vazio
`(S)S`	`$`	reduce $S \to (S)S$	o topo é o lado direito
S	`$`	accept	sobrou S e a entrada acabou

Reduce $S \to \varepsilon$ tira **zero** símbolos e empilha S. Lendo os *reduce* de baixo para cima obtém-se a derivação mais à direita: $S \Rightarrow (S)\underline{S} \Rightarrow (\underline{S}) \Rightarrow ()$.

Exemplo resolvido **Exemplo 2 dos slides: expressões simples**

Gramática: $E' \to E\,\$$ e $E \to E + \mathtt{n} \mid \mathtt{n}$. Entrada `n+n`.

Pilha	Entrada	Ação
ε	n+n$	shift
n	+n$	reduce $E \to \mathtt{n}$
E	+n$	shift
E+	n$	shift
E+n	$	reduce $E \to E + \mathtt{n}$
E	$	accept

Derivação: $E \Rightarrow E + \mathtt{n} \Rightarrow \mathtt{n} + \mathtt{n}$. Com a pilha E+n, **não** se reduz o n a E: daria $E + E$, que não é lado direito de nenhuma produção. O analisador precisa de saber **o que está por baixo** do topo, não só o topo: é isso que motiva os estados.

Tabela de *parsing* LR e o algoritmo

A ação depende da **configuração da pilha** e, possivelmente, dos próximos símbolos da entrada (*look-ahead*). Para não olhar para a pilha toda, **resume-se a pilha num estado** (um inteiro), e as ações ficam numa **tabela**.

Definição **Tabela de parsing LR**

Linhas = **estados**; colunas = **símbolos** (terminais à esquerda, não-terminais à direita). Cada entrada é:

- $s\,q$ (*shift* q): empilha o próximo terminal e vai para q;
- $r\,k$ (*reduce* k), com $X \to \alpha_1 \dots \alpha_n$ a produção k: tira n estados da pilha, olha para o estado que fica no topo, e vai para o estado da entrada "$g\,q$" na coluna X desse estado;
- $g\,q$ (*go* q): muda para q (só nas colunas de não-terminais, depois de um *reduce*);
- a (*accept*): aceita;
- **vazia**: erro de sintaxe.

Exemplo resolvido **Tabela dos slides**

Nº	Produção
(0)	$T' \to T\,\$$
(1)	$T \to R$
(2)	$T \to aTc$
(3)	$R \to \varepsilon$
(4)	$R \to bR$

	a	b	c	$	T	R
0	s3	s4	r3	r3	g1	g2
1				a		
2			r1	r1		
3	s3	s4	r3	r3	g5	g2
4		s4	r3	r3		g6
5			s7			
6			r4	r4		
7			r2	r2		

Leitura: estado 0 com a → *shift*, vai para 3. Estado 3 com c → *reduce* pela regra 3 ($R \to \varepsilon$). Estado 0 depois de um *reduce* para $T \to$ g1. Estado 1 com $ → aceita.

Exemplo resolvido **Análise de aabbbcc com a tabela acima (1/2)**

"Estados" é a **pilha de estados inteira** (o topo é o último número); "Símbolos" é a pilha de símbolos correspondente.

Estados	Símbolos	Entrada	Ação
0	ε	`aabbbcc$`	(0,a) = s3: shift 3
0 3	a	`abbbcc$`	(3,a) = s3: shift 3
0 3 3	a a	`bbbcc$`	(3,b) = s4: shift 4
0 3 3 4	a a b	`bbcc$`	(4,b) = s4: shift 4
0 3 3 4 4	a a b b	`bcc$`	(4,b) = s4: shift 4
0 3 3 4 4 4	a a b b b	`cc$`	(4,c) = r3
0 3 3 4 4 4 6	a a b b b R	`cc$`	(6,c) = r4
0 3 3 4 4 6	a a b b R	`cc$`	(6,c) = r4

- **(4,c) = r3**, $R \to \varepsilon$: tira **0** estados. Topo 4; (4,R) = g6, empilha 6.
- **(6,c) = r4**, $R \to bR$: tira **2** estados, fica `0 3 3 4 4`. Topo 4; (4,R) = g6, empilha 6.
- **(6,c) = r4** outra vez: tira 2, fica `0 3 3 4`; (4,R) = g6, empilha 6.

Exemplo resolvido **Análise de aabbbcc com a tabela acima (2/2)**

Estados	Símbolos	Entrada	Ação
0 3 3 4 6	a a b R	cc$	(6,c) = r4
0 3 3 2	a a R	cc$	(2,c) = r1
0 3 3 5	a a T	cc$	(5,c) = s7: shift 7
0 3 3 5 7	a a T c	c$	(7,c) = r2
0 3 5	a T	c$	(5,c) = s7: shift 7
0 3 5 7	a T c	$	(7,$) = r2
0 1	T	$	(1,$) = a: **accept**

- **(6,c) = r4**: tira 2 de `0 3 3 4 6`, fica `0 3 3`. (3,R) = g2, empilha 2.
- **(2,c) = r1**, $T \to R$: tira 1, fica `0 3 3`. (3,T) = g5, empilha 5.
- **(7,c) = r2**, $T \to aTc$: tira **3** de `0 3 3 5 7`, fica `0 3`. (3,T) = g5, empilha 5.
- **(7,$) = r2**: tira 3 de `0 3 5 7`, fica `0`. (0,T) = g1, empilha 1. Com $, (1,$) = a: aceita.

Definição **Algoritmo de parsing LR**

```
stack = empty; push(0, stack); next = getToken()
loop
  case table[top(stack), next] of
    shift s:  push(s, stack); next = getToken()
    reduce p: let X = lado esquerdo da producao p
                  n = comprimento do lado direito da producao p
              pop n elementos da pilha
              lookup table[top(stack), X] e encontra "go s"
              push(s, stack)
    accept:   termina com sucesso
    vazio:    reporta erro
```

A decisão vem **só da tabela**, e basta guardar **estados** na pilha (cada estado já resume os símbolos). **O difícil é construir a tabela**, que se faz a partir da gramática, antes de qualquer entrada.

Definição **LR(0), LR(1), LR(k)**

Número de símbolos de *look-ahead* que a tabela usa: **LR(0)** nenhum (só a pilha), **LR(1)** um, **LR(k)** k. LR(0) é a mais simples mas **reconhece poucas linguagens**; LR(k) com $k \geq 2$ dá tabelas **muito grandes**; **LR(1) chega para a maior parte das linguagens de programação**.

Autómato e tabela LR(0)

Items LR(0)

Definição **Item LR(0)**

Uma produção com uma **posição marcada** (•) no lado direito. Os **estados** do autómato LR são **conjuntos de items**. $A \to \beta \bullet \gamma$: **$\beta$ já está no topo da pilha**, e falta reconhecer γ.

- $A \to \bullet\gamma$ — **item inicial**: ainda nada reconhecido.
- $A \to \gamma\bullet$ — **item completo**: pode-se reduzir a A.

Exemplo resolvido

Os **9 items** da gramática dos parêntesis (um lado direito de comprimento n dá $n+1$ items; $S \to \varepsilon$ dá só $S \to \bullet$):

Produções	Items
$S' \to S\,\$$	$S' \to \bullet S\,\$ \quad S' \to S \bullet \$ \quad S' \to S\,\$\bullet$
$S \to (S)S$	$S \to \bullet(S)S \quad S \to (\bullet S)S \quad S \to (S\bullet)S$
	$S \to (S) \bullet S \quad S \to (S)S\bullet$
$S \to \varepsilon$	$S \to \bullet$

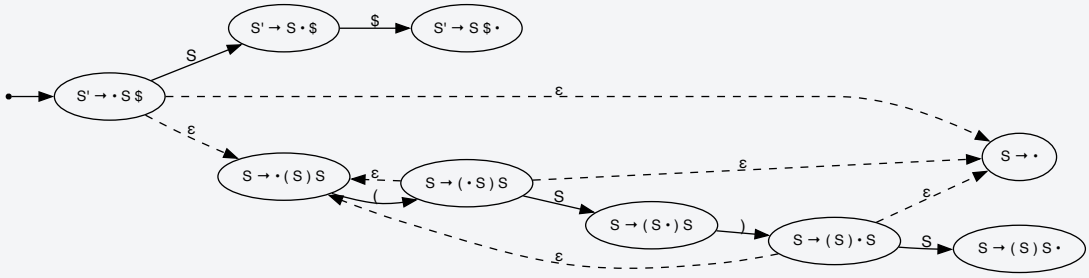
Do NFA de items ao DFA

Definição **Transições entre items**

Um **NFA** cujos estados são os items:

- **por um símbolo** X: de $A \to \alpha \bullet X\gamma$ para $A \to \alpha X \bullet \gamma$. Por um terminal, depois de um **shift**; por um não-terminal, depois de um **reduce** (o *go*).
- ε: de $A \to \alpha \bullet B\gamma$ (B não-terminal) para **todos** os $B \to \bullet\beta$: para avançar sobre B é preciso começar a reconhecer um B.
- **Estado inicial**: $S' \to \bullet S\,\$$. **Não há estados finais**: aceita-se quando se faria *shift* do \$ (ação *accept*).

Exemplo resolvido **NFA de items da gramática dos parêntesis**



Tracejado: transições-ε. De $S \to (\bullet S)S$ sai a transição por S para $S \to (S\bullet)S$, e duas transições-ε para $S \to \bullet(S)S$ e $S \to \bullet$.

O NFA converte-se num **DFA** pela **construção de subconjuntos** (Aula 2), em que cada estado é um **conjunto de items**. Na prática faz-se diretamente, sem desenhar o NFA:

Definição **Fecho e goto (construção direta do DFA)**

- **fecho(I)** (o *ε-closure*): parte dos items de I; para cada item com o ponto antes de um não-terminal B, junta **todos** os $B \to \bullet\beta$; repete até não entrar nada novo.
- **goto(I, X)**: pega nos items de I com o ponto antes de X, avança o ponto e aplica o **fecho**.

Estado inicial: fecho($\{S' \to \bullet S\,\$\}$). Calcula-se goto(I, X) para cada estado e cada símbolo; cada conjunto novo é um estado novo, até não aparecerem mais.

Exemplo resolvido **DFA LR(0) dos parêntesis, passo a passo (1/2)**

Gramática: (0) $S' \to S\,\$$, (1) $S \to (S)S$, (2) $S \to \varepsilon$. Numeração dos estados como nos slides.
Estado 0 = fecho($\{S' \to \bullet S\,\$\}$). O ponto está antes de S: entram $S \to \bullet(S)S$ e $S \to \bullet$. Nestes, o ponto está antes de (ou no fim: não entra mais nada.

$$0 = \{\, S' \to \bullet S\,\$,\;\; S \to \bullet(S)S,\;\; S \to \bullet \,\}$$

Transições de 0 (só (e S aparecem a seguir a um ponto):

- goto(0, (): de $S \to \bullet(S)S$ vem $S \to (\bullet S)S$; o fecho junta $S \to \bullet(S)S$ e $S \to \bullet$. **Estado 2** = $\{S \to (\bullet S)S,\; S \to \bullet(S)S,\; S \to \bullet\}$.
- goto(0, S): de $S' \to \bullet S\,\$$ vem $S' \to S \bullet \$$; ponto antes de \$, o fecho não junta nada. **Estado 1** = $\{S' \to S \bullet \$\}$.

Transições de 1: só por \$, que é o *accept*.

Exemplo resolvido **DFA LR(0) dos parêntesis, passo a passo (2/2)**

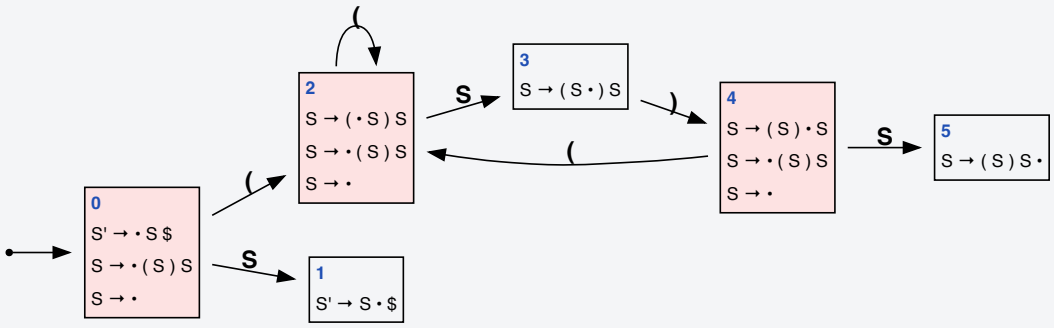
Transições de 2 = $\{S \to (\bullet S)S,\; S \to \bullet(S)S,\; S \to \bullet\}$:

- goto(2, (): de $S \to \bullet(S)S$ vem $S \to (\bullet S)S$, cujo fecho é o **estado 2** outra vez (ciclo).
- goto(2, S): de $S \to (\bullet S)S$ vem $S \to (S\bullet)S$; ponto antes de). **Estado 3** = $\{S \to (S\bullet)S\}$.

Transições de 3: goto(3,)) dá $S \to (S) \bullet S$; ponto antes de S, entram $S \to \bullet(S)S$ e $S \to \bullet$. **Estado 4** = $\{S \to (S) \bullet S,\; S \to \bullet(S)S,\; S \to \bullet\}$.
Transições de 4:

- goto(4, (): de $S \to \bullet(S)S$ vem o **estado 2**.
- goto(4, S): de $S \to (S) \bullet S$ vem $S \to (S)S\bullet$. **Estado 5** = $\{S \to (S)S\bullet\}$.

Transições de 5: nenhuma (só um item completo). Fim: estados 0 a 5.



A rosa: os estados que vão ter conflitos na tabela LR(0).

Construção da tabela LR(0)

Definição **Tabela LR(0)**

1. Transição por um **terminal** t de i para j: $s\,j$ em (i, t).
2. Transição por um **não-terminal** X de i para j: $g\,j$ em (i, X).
3. Estado i com um **item completo** $A \to \gamma\bullet$ (produção k): $r\,k$ em **todas** as colunas de terminais da linha i (LR(0) não olha para o próximo símbolo).
4. Estado com $S' \to S \bullet \$$, coluna \$: **accept**.

A gramática é **LR(0)** se **cada entrada tem no máximo uma ação**. Duas ações na mesma entrada são um **conflito**.

Exemplo resolvido **Tabela LR(0) dos parêntesis**

1. *Shifts*: s2 em (0,(), (2,(), (4,(); s4 em (3,)).
2. *Gotos*: g1, g3, g5 na coluna S das linhas 0, 2, 4.
3. *Reduces*: 0, 2 e 4 têm $S \to \bullet$ (produção 2): r2 em todas as colunas terminais. O 5 tem $S \to (S)S\bullet$ (produção 1): r1 em todas.
4. *Accept*: (1, $).

	(	)	$	S
0	**s2, r2**	r2	r2	g1
1			a	
2	**s2, r2**	r2	r2	g3
3		s4		
4	**s2, r2**	r2	r2	g5
5	r1	r1	r1	

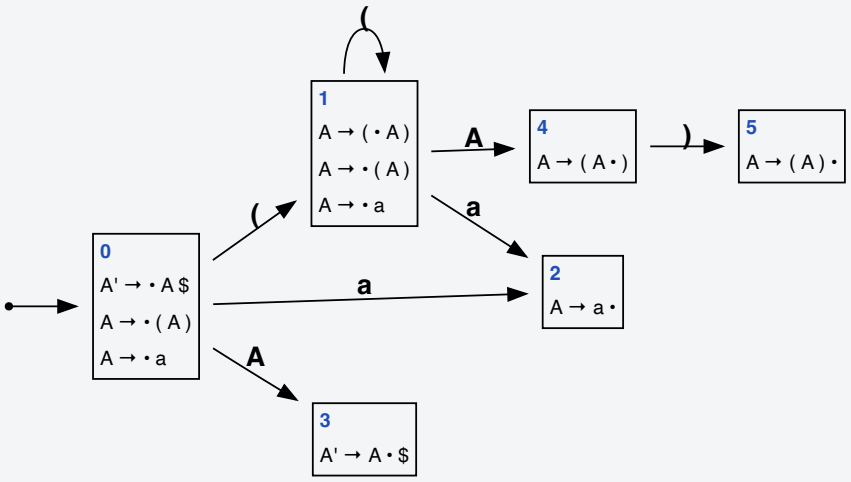
Em 0, 2 e 4 com (há **duas ações**: *shift* (abrir um par novo) ou *reduce* $S \to \varepsilon$. São **conflitos shift/reduce**: a gramática **não é LR(0)** (resolve-se com SLR(1), abaixo).

Exemplo resolvido **Exercício 3(a) (Folha lab. 3) = Exercício 1 dos slides: construção**

Mostrar que $A \to (A) \mid \mathtt{a}$ é LR(0). Gramática aumentada: (0) $A' \to A\,\$$, (1) $A \to (A)$, (2) $A \to \mathtt{a}$.
Estado 0 = fecho($\{A' \to \bullet A\,\$\}$): ponto antes de A, entram $A \to \bullet(A)$ e $A \to \bullet\mathtt{a}$. $0 = \{A' \to \bullet A\,\$,\ A \to \bullet(A),\ A \to \bullet\mathtt{a}\}$.

- goto(0, () = fecho($\{A \to (\bullet A)\}$) = $\{A \to (\bullet A),\ A \to \bullet(A),\ A \to \bullet\mathtt{a}\}$: **estado 1**.
- goto(0, a) = $\{A \to \mathtt{a}\bullet\}$: **estado 2**.
- goto(0, A) = $\{A' \to A \bullet \$\}$: **estado 3**.

Transições de 1: goto(1, () = **estado 1**; goto(1, a) = **estado 2**; goto(1, A) = $\{A \to (A\bullet)\}$: **estado 4**.
De 2 (só item completo) e **de 3** (só $, accept): nenhuma.
De 4: goto(4,)) = $\{A \to (A)\bullet\}$: **estado 5**, sem transições.



Exemplo resolvido **Exercício 3(a) (Folha lab. 3): tabela e verificação**

Shifts: s1 em (0,() e (1,(); s2 em (0,a) e (1,a); s5 em (4,)). *Gotos*: g3 em (0,*A*), g4 em (1,*A*). *Reduces*: estado 2 (produção 2) r2 em todas as colunas terminais; estado 5 (produção 1) r1 em todas. *Accept*: (3,$).

	(	)	a	$	A
0	s1		s2		g3
1	s1		s2		g4
2	r2	r2	r2	r2	
3				a	
4		s5			
5	r1	r1	r1	r1	

Nenhuma entrada tem duas ações (os estados com *reduce* não têm *shift*): a gramática **é LR(0)**. Teste com `((a))`:

Estados	Símbolos	Entrada	Ação
0	ε	((a))$	s1
0 1	(	(a))$	s1
0 1 1	((	a))$	s2
0 1 1 2	((a	))$	r2: tira 1, topo 1, (1,A) = g4
0 1 1 4	((A	))$	s5
0 1 1 4 5	((A)	)$	r1: tira 3, topo 1, (1,A) = g4
0 1 4	(A	)$	s5
0 1 4 5	(A)	$	r1: tira 3, topo 0, (0,A) = g3
0 3	A	$	accept

Pratica agora **Autómato e tabela LR(0) (Folha lab. 3, Exercícios 4 e 5)**

Ex. 4 Gramática $E \to (L) \mid \texttt{a}$, $L \to L, E \mid E$.

- **(a)** Derivação de `((a),a,(a,a))`. Usa as derivações da Aula 4; escolhe sempre o não-terminal mais à esquerda para não te perderes.
- **(b)** Autómato LR(0). Atenção ao estado depois de `(`: tem o ponto antes de L, e $L \to \bullet L, E$ também tem o ponto antes de L.
- **(c)** É LR(0)? Procura estados com um item completo **e** outro item (com *shift* ou outro *reduce*).

Ex. 5 Declarações de C: $Decl \to Type\ Varlist$;, $Type \to$ `int` | `float`, $Varlist \to Varlist$, `ident` | `ident`.

- **(a)** Autómato e tabela LR(0), como no Exercício 3(a) acima.
- **(b)** Simula `int x,y,z;` com a pilha de estados, como no teste de `((a))` acima (o analisador lexical dá `int ident , ident , ident ;`).

Soluções: Ex. 4 (a), Ex. 4 (b), Ex. 4 (c), Ex. 5 (a), Ex. 5 (b)

Análise SLR(1)

Muitas gramáticas úteis **não são LR(0)** (a dos parêntesis já não era). Usando também o **próximo terminal** (*look-ahead*) reconhecem-se muito mais. A extensão mais simples é a **SLR(1)** (*Simple LR*), que usa o conjunto FOLLOW.

FIRST e FOLLOW

Definição **Anulável, FIRST e FOLLOW †**

- **Anulável**: $X \Rightarrow^* \varepsilon$ (ex.: há $X \to \varepsilon$, ou $X \to YZ$ com Y e Z anuláveis).
- **FIRST(α)**: os terminais que podem **começar** uma palavra derivada de $\alpha = X_1 \dots X_n$: FIRST(X_1), mais FIRST(X_2) se X_1 for anulável, e assim por diante. FIRST(t) = $\{t\}$ para um terminal.
- **FOLLOW(A)**: os terminais que podem aparecer **logo a seguir** a A. Como $S' \to S\,\$$, o \$ está sempre em FOLLOW(S).

Cálculo de FOLLOW: começa vazio; para cada ocorrência $X \to \alpha A \beta$ de um não-terminal A:

1. junta FIRST(β) a FOLLOW(A);
2. se β é vazio ou anulável, junta **FOLLOW(X)** a FOLLOW(A).

Repete até nenhum conjunto mudar.

Exemplo resolvido **FOLLOW da gramática dos parêntesis**

Produções: $S' \to S\,\$$, $S \to (S)S$, $S \to \varepsilon$. Ocorrências de S:

1. $S' \to \underline{S}\,\$$: FIRST(\$) = {\$}, logo \$ ∈ FOLLOW(S).
2. $S \to (\underline{S})S$: FIRST()S) = {)}, logo) ∈ FOLLOW(S).
3. $S \to (S)\underline{S}$: no fim, junta FOLLOW(S) a si próprio: nada.

Segunda passagem: nada muda. **FOLLOW(S) = {), \$}**: um S acaba antes de um) ou no fim da entrada.

Exemplo resolvido **FOLLOW das expressões aritméticas (Aula 4)**

$E' \to E\,\$$, $E \to E+T \mid T$, $T \to T*F \mid F$, $F \to \texttt{num} \mid (E)$. Nada é anulável, e FIRST(F) = FIRST(T) = FIRST(E) = {num, (}.

- E em $E' \to \underline{E}\,\$$ (\$), $E \to \underline{E} + T$ (+) e $F \to (\underline{E})$ ()). FOLLOW(E) = {\$, +,)}.
- T em $E \to E + \underline{T}$ e $E \to \underline{T}$ (no fim: junta FOLLOW(E)) e em $T \to \underline{T} * F$ (∗). FOLLOW(T) = {\$, +,), ∗}.
- F em $T \to T * \underline{F}$ e $T \to \underline{F}$, sempre no fim: junta FOLLOW(T). FOLLOW(F) = {\$, +,), ∗}.

Segunda passagem: nada muda.

Tabela SLR(1)

Definição **Tabela SLR(1)**

Mesmo **algoritmo** e mesmo **autómato** do LR(0). Só muda a tabela: *shift* e *goto* como antes, mas o *reduce* de um item completo $A \to \gamma\bullet$ só vai para as **colunas dos terminais em FOLLOW(A)**. Só faz sentido reduzir para A se o próximo símbolo puder vir a seguir a um A. A gramática é **SLR(1)** se a tabela não tiver conflitos.

Exemplo resolvido **Tabela SLR(1) dos parêntesis**

Mesmo DFA (estados 0 a 5), mesmos *shifts* e *gotos*. Os *reduce* vão só para FOLLOW(S) = {), \$}:

- 0, 2 e 4 ($S \to \bullet$, produção 2): r2 em) e \$, **não** em (;
- 5 ($S \to (S)S\bullet$, produção 1): r1 em) e \$.

	(	)	\$	S
0	s2	r2	r2	g1
1			a	
2	s2	r2	r2	g3
3		s4		
4	s2	r2	r2	g5
5		r1	r1	

Sem conflitos: a gramática **é SLR(1)** (sem ser LR(0)). O conflito em (desaparece porque (não está em FOLLOW(S): com (à frente, um S vazio nunca estaria certo, e faz-se *shift*.

Exemplo resolvido **Análise de (()) com a tabela SLR(1)**

Estados	Símbolos	Entrada	Ação
0	ε	(())\$	s2
0 2	(	())\$	s2
0 2 2	((	))\$	r2: tira 0, topo 2, (2,S) = g3
0 2 2 3	((S	))\$	s4
0 2 2 3 4	((S)	)\$	r2: tira 0, topo 4, (4,S) = g5
0 2 2 3 4 5	((S) S	)\$	r1: tira 4, topo 2, (2,S) = g3
0 2 3	(S	)\$	s4
0 2 3 4	(S)	\$	r2: tira 0, topo 4, (4,S) = g5
0 2 3 4 5	(S) S	\$	r1: tira 4, topo 0, (0,S) = g1
0 1	S	\$	accept

No primeiro passo, estado 0 com (, onde a tabela LR(0) tinha conflito, a SLR(1) diz s2 sem hesitar.

Pratica agora **SLR(1) (Folha lab. 3, Exercício 3(b)) = Exercício 2 dos slides**

Ex. 3(b) Mostra que $T \to R$, $T \to aTc$, $R \to \varepsilon$, $R \to bR$ é SLR(1): constrói o autómato e a tabela. Justifica que **não** é LR(0) mostrando os conflitos da tabela LR(0).
Pista: é a gramática da tabela dos slides (secção "Tabela de *parsing* LR"), por isso podes comparar no fim. Os slides avisam que a tua numeração dos estados pode sair diferente, mas sem conflitos. Calcula FOLLOW(T) e FOLLOW(R) primeiro (a produção $T \to R$ faz com que FOLLOW(T) entre em FOLLOW(R)).
Soluções: Ex. 3 (b)

Conflitos e como resolvê-los

Definição **Conflito**

Uma entrada da tabela com **duas ou mais ações**.

- **shift/reduce**: um *shift* e um *reduce* no mesmo estado e símbolo.
- **reduce/reduce**: dois *reduce* no mesmo estado e símbolo.

Os conflitos **podem** vir de ambiguidade, **mas nem sempre**: a gramática dos parêntesis não é ambígua e tinha conflitos LR(0). Já uma gramática ambígua dá **sempre** conflitos (duas árvores querem dizer que, nalgum passo,

duas ações levam à aceitação). Os geradores de analisadores (Aula 6) avisam de todos os conflitos. Formas de os eliminar:

- **reescrever a gramática** (Aula 4);
- declarar **associatividades e precedências** dos *tokens* (Aula 6);
- escolher ***shift*** em vez de *reduce* (o que os geradores fazem por omissão).

Exemplo resolvido **Dangling else num analisador SLR(1)**

$S \to$ `if cond then` S `else` $S \mid$ `if cond then` $S \mid$ `skip`. Há um estado com estes dois items:

$$S \to \texttt{if cond then } S \bullet \qquad\qquad S \to \texttt{if cond then } S \bullet \texttt{ else } S$$

Com `else` à frente, o analisador pode:

- **shift** do `else` (segundo item);
- **reduce** pela produção sem `else` (primeiro item), porque $\texttt{else} \in \text{FOLLOW}(S) = \{\texttt{else}, \$\}$.

Em `if cond then if cond then skip else skip`, com a pilha `if cond then if cond then S` e `else` à frente:

- **shift**: o `else` junta-se ao `if` **de dentro**: `if cond then { if cond then skip else skip }`;
- **reduce**: fecha o `if` de dentro sem `else`, e o `else` fica para o de fora: `if cond then { if cond then skip } else skip`.

Escolhendo *shift* obtém-se a interpretação usual (Pascal, C, Java): o `else` liga-se ao `if` mais próximo. Por isso o *shift* por omissão resolve este caso sem mexer na gramática.

Extras: LR(1) e LALR(1)

A SLR(1) **não chega para algumas construções das linguagens de programação**. A **LR(1)** resolve essas limitações, mas pode ter **muito mais estados**. Na prática os geradores usam a **LALR(1)** (*Look-Ahead LR(1)*), mais compacta. As diferenças são técnicas: **percebendo SLR(1) já se percebe e usa um gerador**.

Definição **Items e autómato LR(1)**

Em LR(1) o *look-ahead* entra na **definição dos estados**, para os distinguir e evitar conflitos.

- **Item LR(1)**: $(A \to \alpha \bullet \beta,\ a)$, um item LR(0) e um terminal a. Quer dizer: α está no topo da pilha e o resto da entrada deriva de βa. Os estados são **conjuntos** destes items.
- **Por um símbolo** X: de $(A \to \alpha \bullet X\gamma,\ a)$ para $(A \to \alpha X \bullet \gamma,\ a)$ (o *look-ahead* não muda).
- ε: de $(A \to \alpha \bullet B\gamma,\ a)$ para $(B \to \bullet\beta,\ b)$, para **todas** as produções $B \to \beta$ e **todos** os $b \in \text{FIRST}(\gamma a)$.
- **Tabela**: *shift* e *go* como antes; *reduce* $A \to \alpha$ quando o estado tem $(A \to \alpha\bullet,\ a)$ **e o próximo terminal é** a.

Exemplo resolvido **O início do autómato LR(1) dos parêntesis †**

Gramática $S' \to S\,\$$, $S \to (S)S \mid \varepsilon$.
Estado inicial. De $S' \to \bullet S\,\$$, o ponto está antes de S com $\gamma = \$$: $\text{FIRST}(\$) = \{\$\}$. Entram $(S \to \bullet(S)S,\ \$)$ e $(S \to \bullet,\ \$)$.
Depois de `(`. De $(S \to \bullet(S)S,\ \$)$ vem $(S \to (\bullet S)S,\ \$)$. Ponto antes de S, com $\gamma =)S$ e $a = \$$: $\text{FIRST}()S\$) = \{)\}$. Entram $(S \to \bullet(S)S,\ ))$ e $(S \to \bullet,\ ))$.
Aqui o *reduce* $S \to \varepsilon$ faz-se **só com `)`** (em SLR(1) era com `)` e \$). Um segundo `(` vai para um estado novo, com $(S \to (\bullet S)S,\ ))$: outro *look-ahead*. Assim a LR(1) distingue mais situações e tem mais estados. A **LALR(1)** volta a juntar os estados LR(1) com os mesmos items LR(0), e fica com o número de estados do LR(0), mas com *look-aheads* mais precisos do que o FOLLOW.

6 Aula 6
Geradores de analisadores sintáticos e sintaxe abstrata

Construir a tabela LR à mão só é viável para gramáticas pequenas; para uma linguagem real usa-se um **programa que a gera**. Esta aula tem duas partes: os **geradores** *Happy* (Haskell) e *Bison* (C), com **ações semânticas** e resolução de **conflitos**; e a **sintaxe abstrata** (a AST), que é a saída da análise sintática.

Geradores de analisadores LR

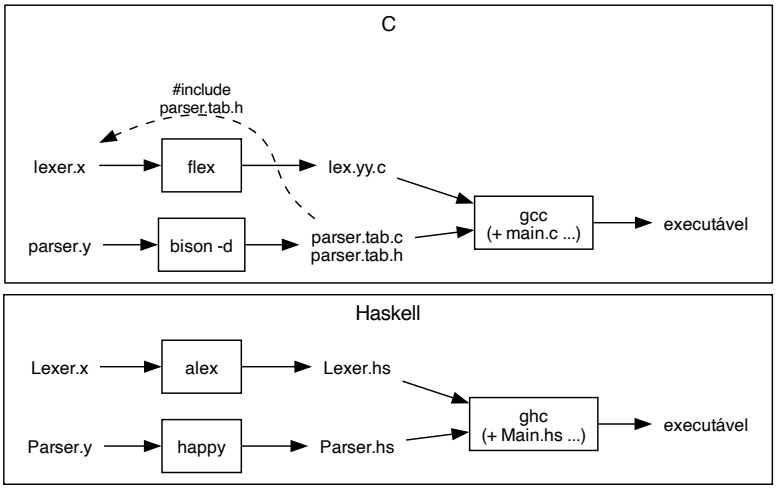
Definição **Gerador de analisadores sintáticos**

Recebe uma **gramática** (com ações em cada produção) e produz o **código-fonte** de um analisador LR para ela: o autómato, a tabela e o ciclo de *parsing* da Aula 5.

- **Yacc** (*Yet Another Compiler Compiler*): o original do UNIX, para C. Gera analisadores **LALR(1)**.
- **Bison**: reimplementação GNU do Yacc, para C ou C++.
- **Happy**: semelhante, gera analisadores **LALR(1)** em Haskell.

Também há geradores LL (*top-down*), que não vamos usar.

Cada gerador trabalha com um gerador lexical da Aula 3: **Happy + Alex** em Haskell, **Bison + Flex** em C.




```
$ alex Lexer.x                      $ bison -d parser.y
$ happy Parser.y                    $ flex lexer.x
$ ghc -o prog Lexer.hs Parser.hs ...  $ gcc -o prog parser.tab.c lex.yy.c ...
```

Em C, o `-d` do Bison gera o *header* `parser.tab.h` com os códigos dos *tokens*, que o `lexer.x` inclui para devolver os mesmos números. Em Haskell esse papel é do tipo `Token`, definido no `Lexer.x` e importado pelo `Parser.y`.

Happy

Definição **Happy**

Gerador **LALR(1)** para Haskell, integrado com o **Alex**. Recebe um ficheiro (`Parser.y`) com a **gramática anotada**: produções com **ações semânticas** (expressões Haskell entre chavetas), **diretivas** (`%name`, `%token`, …) e **código auxiliar** Haskell. Gera `Parser.hs`.

Exemplo resolvido **Um reconhecedor em Happy (slides)**

Expressões com + e parêntesis, já desambiguadas como na Aula 4:

```
{                                   -- (1) código Haskell do início
module Parser where
import Lexer
}

%name parser                        -- (2) diretivas
%tokentype { Token }
%error { parseError }

%token                              -- (3) tokens
num         { TOK_NUM $$ }
'+'         { TOK_PLUS }
'('         { TOK_LPAREN }
')'         { TOK_RPAREN }

%%                                  -- (4) gramática

Exp : Term                  { () }
    | Exp '+' Term          { () }

Term : num                  { () }
     | '(' Exp ')'          { () }

{                                   -- (5) código Haskell do fim
parseError :: [Token] -> a
parseError toks = error "parse error"
}
```

No `Lexer.x`: `data Token = TOK_NUM Int | TOK_PLUS | TOK_LPAREN | TOK_RPAREN`.

- **(1)** e **(5)**: Haskell copiado para o `Parser.hs`. A `parseError` é chamada num erro sintático (entrada vazia da tabela).
- **(2)**: `%name parser` é o **nome da função** gerada; `%tokentype` o tipo dos *tokens*; `%error` a função de erro.
- **(3)**: `%token` liga cada **terminal** (à esquerda) ao **padrão Haskell** do *token* (à direita). O `$$` marca o **valor semântico** do *token*: o valor de `TOK_NUM 42` é `42`. Sem `$$`, o valor é o próprio *token*.
- **(4)**: as produções, com `:` no lugar de →. **O primeiro não-terminal é o símbolo inicial.**

Aqui as ações devolvem todas `()`, por isso `parser :: [Token] -> ()`: só **reconhece**. O programa principal:

```haskell
module Main where
import Parser
import Lexer

main :: IO ()
main = do
  txt <- getContents
  print (parser $ alexScanTokens txt)
```

Imprime `()` se a entrada respeita a gramática; senão `parseError` lança uma exceção.

Ações semânticas

Além de reconhecer, um **compilador** constrói uma **representação do programa** (a árvore) e um **interpretador executa-o**. Ambos se fazem com **ações semânticas** em cada produção.

> **Definição** **Ação semântica e valor semântico**
>
> Cada símbolo tem um **valor semântico**: o de um **terminal** vem do `%token` (o `$$`); o de um **não-terminal** é o resultado da ação da sua produção. Numa ação, **`$1, $2, $3, …`** são os valores dos símbolos do **lado direito**, por ordem: em `Exp : Exp '+' Term { Add $1 $3 }`, `$1` é o `Exp`, `$2` o `'+'` e `$3` o `Term`.
> **A ação corre no *reduce*** dessa produção †: os símbolos do lado direito estão no topo da pilha, cada um com o seu valor; a ação calcula o valor do lado esquerdo, que é empilhado no lugar deles. Por isso as ações correm **de baixo para cima** na árvore: as sub-árvores antes do nó pai.

Construir a árvore sintática

Declara-se um **tipo algébrico** para as árvores e cada ação monta o nó a partir das sub-árvores:

```
Exp : Term                { $1 }
    | Exp '+' Term        { Add $1 $3 }

Term : num                { Num $1 }
     | '(' Exp ')'        { $2 }

{
data Exp = Num Int
         | Add Exp Exp
         deriving Show
}
```

- `Exp : Term { $1 }`: a árvore é a do termo, sem nó novo.
- `Exp '+' Term { Add $1 $3 }`: nó `Add` com as duas sub-árvores; o `'+'` não entra, o construtor já diz que é uma soma.
- `num { Num $1 }`: `$1` é o `Int` dentro do `TOK_NUM`.
- `'(' Exp ')' { $2 }`: os parêntesis **desaparecem**; a forma da árvore já guarda o agrupamento.

A função passa a `parser :: [Token] -> Exp`, e o resultado já é uma **árvore sintática abstrata** (ver abaixo).

Calcular o resultado

Em vez de construir a árvore, **calcula-se logo o valor**:

```
Exp : Term                { $1 }
    | Exp '+' Term        { $1 + $3 }

Term : num                { $1 }
     | '(' Exp ')'        { $2 }
```

`parser :: [Token] -> Int` é um **interpretador**: recebe os *tokens* e calcula o valor (ou lança uma exceção num erro sintático).

Exemplo resolvido **Os valores na pilha ao analisar (1+2)+3**

Ações do interpretador; cada símbolo aparece como `símbolo:valor`. As decisões *shift/reduce* são as da tabela LALR(1) do Happy.

#	Pilha	Entrada	Ação	Valor
1		`(1+2)+3`	shift `(`	
2	`(`	`1+2)+3`	shift `num`	*token*: 1
3	`( num:1`	`+2)+3`	reduce `Term : num`	`$1` = 1
4	`( Term:1`	`+2)+3`	reduce `Exp : Term`	`$1` = 1
5	`( Exp:1`	`+2)+3`	shift `'+'`	
6	`( Exp:1 +`	`2)+3`	shift `num`	*token*: 2
7	`( Exp:1 + num:2`	`)+3`	reduce `Term : num`	`$1` = 2
8	`( Exp:1 + Term:2`	`)+3`	reduce `Exp : Exp '+' Term`	1+2 = 3
9	`( Exp:3`	`)+3`	shift `')'`	
10	`( Exp:3 )`	`+3`	reduce `Term : '(' Exp ')'`	`$2` = 3
11	`Term:3`	`+3`	reduce `Exp : Term`	`$1` = 3
12	`Exp:3`	`+3`	shift `'+'`	
13	`Exp:3 +`	`3`	shift `num`	*token*: 3
14	`Exp:3 + num:3`		reduce `Term : num`	`$1` = 3
15	`Exp:3 + Term:3`		reduce `Exp : Exp '+' Term`	3+3 = 6
16	`Exp:6`		accept	**6**

No passo 8 o *reduce* tira os três símbolos do topo: `Exp:1` é `$1`, `+` é `$2`, `Term:2` é `$3`, e a ação dá `$1 + $3`. No passo 10, `(` é `$1`, `Exp:3` é `$2` e `)` é `$3`: a ação é `$2`.

Com as ações que constroem a árvore, os *shift/reduce* são os mesmos (a tabela só depende da gramática); só mudam os valores: passos 3–4 `Num 1`; 7 `Num 2`; 8 `Add (Num 1) (Num 2)`; 10–11 o mesmo; 14 `Num 3`; 15 `Add (Add (Num 1) (Num 2)) (Num 3)`, que é o resultado.

Pratica agora **Ações semânticas (exercício inventado)**

Ex. A Com a gramática `Exp : Term | Exp '+' Term`, `Term : num | '(' Exp ')'` desta secção:

- **(a)** Faz a tabela "pilha / entrada / ação / valor" para `2+(3+4)`, com as ações do interpretador (`$1 + $3`, …). Diz em cada *reduce* quem são `$1`, `$2`, `$3`.
- **(b)** Que valor devolve `parser` para a mesma entrada com as ações que constroem a árvore (`Add $1 $3`, `Num $1`, `$2`)? Compara a forma da árvore com a de `(1+2)+3` no exemplo acima.

Soluções: Ex. A (a), Ex. A (b)

Bison

Definição **Bison**

Gerador LR do projeto GNU para C e C++, semelhante ao Yacc (e ao Happy), integrado com o **Flex**.

Exemplo resolvido **Uma calculadora em Bison (slides)**

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex (void);
void yyerror (char const *);
%}
%define api.value.type {int}
%token TOK_NUM

%%

top : exp            { printf("%d\n", $1); }
    ;

exp :  term          { $$ = $1; }
    | exp '+' term   { $$ = $1 + $3; }
    ;

term : TOK_NUM       { $$ = $1; }
     | '(' exp ')'   { $$ = $2; }
     ;

%%

void yyerror (char const *msg) {
  printf("parse error: %s\n", msg);
  exit(-1);
}

int main(void) {
  yyparse();
}
```

A estrutura é a do Flex: `%{ código C %}` e declarações, `%%`, regras, `%%`, código C. Em relação ao Happy:

- cada **ação é um bloco C**; o resultado dá-se por **atribuição a `$$`** (no Happy é o valor da expressão); cada regra acaba em `;`;
- o **tipo dos valores** declara-se com `%define api.value.type {int}` (o mesmo para todos os símbolos; por omissão `int`);
- o **valor de um terminal** é posto pelo *lexer* na global **`yylval`** antes de devolver o *token*, ex.: `[0-9]+ { yylval = atoi(yytext); return TOK_NUM; }`;
- a função gerada é `yyparse()`, que chama `yylex()` para cada *token*, e `yyerror` em caso de erro;
- a regra `top` é o símbolo inicial (a primeira regra): a sua ação corre uma vez, no último *reduce*, e imprime o resultado.

Terminais de um carácter †, como `'+'` e `'('`, usam-se na gramática sem os declarar; o código do *token* é o código ASCII, por isso o Flex pode fazer `return yytext[0];`. Os *tokens* com nome (`%token TOK_NUM`) recebem números a partir de 258. O fim da entrada é o *token* 0.

	Happy	Bison
Linguagem gerada	Haskell (`Parser.hs`)	C (`parser.tab.c`, `.h` com `-d`)
Analisador lexical	Alex	Flex
Ação semântica	expressão Haskell	bloco C com `$$ = ...;`
Tipo dos valores	um por não-terminal (inferido)	`%define api.value.type` ou `%union`
Valor dos terminais	`$$` no padrão de `%token`	o *lexer* escreve em `yylval`
Função gerada	nome de `%name`, recebe `[Token]`	`yyparse()`, chama `yylex()`
Símbolo inicial	primeiro não-terminal	primeira regra
Erros	função de `%error`	`yyerror`
Relatório do autómato	`happy -i` → `Parser.info`	`bison --report=all` → `parser.output`

Conflitos, associatividade e precedência

Os geradores constroem a tabela LALR(1) e **reportam conflitos** (Aula 5) se a gramática for ambígua (ou não for LALR(1)). Com a gramática ambígua da Aula 4:

```
Exp : num               { ... }
    | Exp '+' Exp       { ... }
    | Exp '*' Exp       { ... }

$ happy Parser.y
shift/reduce conflicts:  4
```

Os dois geradores escrevem um **relatório** com os estados e os conflitos: `happy -i Parser.y` dá `Parser.info`, `bison --report=all parser.y` dá `parser.output`.

Exemplo resolvido **De onde vêm os 4 conflitos (relatório real do Happy) †**

Os conflitos estão em dois estados (o Happy escreve `.` em vez de •):

```
State 6
    Exp -> Exp . '+' Exp        (rule 2)
    Exp -> Exp . '*' Exp        (rule 3)
    Exp -> Exp '*' Exp .        (rule 3)

    '+'     shift, and enter state 4
            (reduce using rule 3)
    '*'     shift, and enter state 5
            (reduce using rule 3)
    %eof    reduce using rule 3
```

No estado 6 a pilha acaba em `Exp * Exp`. O item completo diz "reduz"; os outros dizem que o último `Exp` pode começar um `Exp + Exp` ou `Exp * Exp` maior, e então faz-se *shift* do operador. Com `+` à frente há conflito, com `*` também: 2. O estado 7 é igual com `Exp -> Exp '+' Exp .` (regra 2): mais 2. **Total: 2 estados × 2 operadores = 4.** Com `%eof` só se pode reduzir. A linha entre parêntesis é a ação que **perdeu**: o gerador **escolheu *shift***.

Por omissão, Happy e Bison resolvem os conflitos *shift/reduce* com ***shift*** (e avisam), o que **pode não ser o correto**. Sem declarações, `2*3+4` dá **14** † : com a pilha `Exp * Exp` e `+` à frente, faz *shift* em vez de fechar `2*3`; no fim reduz `3+4 = 7` e depois `2*7 = 14`. O *shift* sistemático faz **todos os operadores associarem à direita, com a mesma precedência**. As saídas são **reescrever a gramática** (Aula 4) ou **declarar associatividade e precedência** (iguais no Happy e no Bison).

Definição **Declarações de associatividade e precedência**

Antes do `%%`:

```
%nonassoc '<' '>' '=='          -- precedência mais baixa
%left '+' '-'
%left '*' '/'                   -- precedência mais elevada
```

- Associatividade: `%left` (esquerda), `%right` (direita) ou `%nonassoc` (não associativo).
- **A ordem das linhas dá a precedência**: cada linha tem **mais** do que as anteriores; *tokens* na mesma linha têm a mesma.

Regra que o gerador aplica † num conflito entre **reduzir pela regra** R e **fazer *shift* do *token*** a:

1. a precedência de R é a do seu **último terminal** (em `Exp : Exp '+' Exp`, a do `'+'`);
2. a com **mais** precedência → *shift*; com **menos** → *reduce*; com **a mesma** → `%left` *reduce*, `%right` *shift*, `%nonassoc` **erro sintático**;
3. se a regra ou o *token* não tiverem precedência, fica o *shift* e o conflito continua a ser reportado.

Exemplo resolvido **Associatividade: 1+2+3 (um só operador)**

Depois de `1+2`, pilha `Exp + Exp`, próximo `+`:

```
Exp -> Exp . '+' Exp        (rule 2)
Exp -> Exp '+' Exp .        (rule 2)

'+'     shift, and enter state 6
        (reduce using rule 2)
```

Regra e *token* têm a precedência do `'+'`: decide a associatividade.

- `%left '+'` → ***reduce***: $(1 + 2) + 3$.
- `%right '+'` → ***shift***: $1 + (2 + 3)$.
- `%nonassoc '+'` → `1+2+3` é **erro sintático** (é o que se quer para `<`: `1<2<3` não deve ser aceite).

Exemplo resolvido **Precedência: 1+2*3 (dois operadores)**

Depois de `1+2`, próximo `*`:

```
Exp -> Exp . '+' Exp        (rule 2)
Exp -> Exp '+' Exp .        (rule 2)
Exp -> Exp . '*' Exp        (rule 3)

'*'     shift, and enter state 7
        (reduce using rule 2)
```

A regra 2 tem a precedência do `'+'` e o *token* é `'*'`. Com `%left '+'` **antes** de `%left '*'`, o `*` tem mais precedência → ***shift***: $1 + (2 \times 3)$.
No caso inverso, `2*3+4`, a pilha é `Exp * Exp` (precedência do `*`) e o *token* `+` tem menos → ***reduce***: $(2\times3)+4 = 10$.
Com estas duas declarações o relatório deixa de ter conflitos.

Dangling else. Nem sempre é preciso eliminar o conflito. No *dangling else* (Aula 5) o relatório mostra:

```
Stm -> if Exp then Stm .              (rule 1)
Stm -> if Exp then Stm . else Stm     (rule 2)

else    shift, and enter state 15
        (reduce using rule 1)
```

O *shift* por omissão liga o `else` ao `if` mais próximo, que é o correto: o gerador avisa, mas o analisador fica certo.

Em resumo, para cada conflito pergunta que árvore dá cada ação e qual queres; depois reescreve a gramática, declara `%left`/`%right`/`%nonassoc` (operadores binários) ou deixa o *shift* (só quando é o que queres), e confirma no relatório que os conflitos desapareceram.

Pratica agora **Precedências no papel (exercício inventado)**

Ex. B Gramática ambígua `Exp : num | Exp '+' Exp | Exp '-' Exp | Exp '*' Exp | Exp '<' Exp`, com a regra acima. Diz que árvore é escolhida e que valor dá:

- **(a)** `8-4-2` com `%right '-'`.
- **(b)** `1+2*3` com as declarações por esta ordem: `%nonassoc '<'`, `%left '*'`, `%left '+' '-'` (repara na ordem!).
- **(c)** `1<2+3` e `1<2<3`, com as mesmas declarações da (b). Para a comparação, conta verdadeiro como 1 e falso como 0.

Soluções: Ex. B (a), Ex. B (b), Ex. B (c)

Pratica agora **Calculadora com Flex/Bison ou Alex/Happy (Folha lab. 4, Exercício 1)**

Ficheiros em `Praticas/Semana_3/Parsing/` (`srcC` ou `srcHaskell`, e os testes em `testsC`/`testsHaskell`). Em C precisas do Bison 3 (`brew install bison`, fica em `/opt/homebrew/opt/bison/bin`): o do macOS é o 2.3 e não aceita `%define api.value.type`. O Happy e o Alex instalam-se com `cabal install happy alex`.

Ex. 1(a) Completa o *lexer* e o *parser* para a gramática ambígua $E \to E{+}E \mid E{-}E \mid E{*}E \mid E/E \mid (E) \mid \texttt{num}$, calculando o valor durante a análise (como em "Calcular o resultado"). Resolve as ambiguidades com `%left` (dois níveis de precedência). Usa vírgula flutuante (`float`/`Float`): o `num` tem de aceitar `1.5`. Confirma `1+2*3` $= 7$, `1+2/3` ≈ 1.667, `1-2-3` $= -4$ e `1/2/3` ≈ 0.167, e corre os testes.

Ex. 1(b) Acrescenta $E \to F(E)$ com $F \to$ `sqrt` | `exp` | `log`. Pistas: as palavras `sqrt`/`exp`/`log` são *tokens* novos no *lexer* (a ordem das regras importa? há identificadores aqui?). Em C, `sqrtf`, `expf`, `logf` de `<math.h>` (o `Makefile` já liga com `-lm`). O `input5.txt` testa isto.

Soluções: 1 (a), 1 (b)

Sintaxe concreta e abstrata

A árvore de derivação (Aula 4) tem detalhes que **não interessam** ao resto do compilador: **parêntesis** de agrupamento, **palavras reservadas** que delimitam blocos (`if`/`then`/`else`, `begin`/`end`) e **não-terminais que só servem para desambiguar** (o T e o F das expressões).

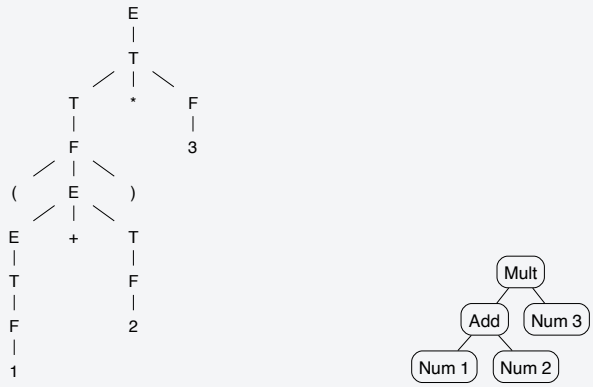
Definição **Sintaxe concreta vs. sintaxe abstrata**

- **Árvore sintática concreta** (de derivação): um nó por cada símbolo de cada produção usada.
- **Árvore sintática abstrata** (AST): só a **estrutura essencial**, com **um nó por operação** (ou construção da linguagem) e os operandos como filhos.

As ações semânticas constroem logo a AST, e a árvore concreta nunca chega a existir em memória.

Exemplo resolvido **(1+2)*3 com a gramática desambiguada**

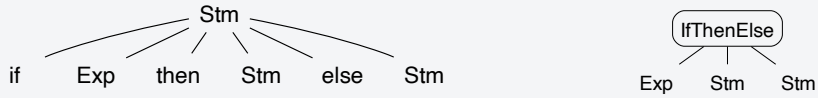
$E \to E + T \mid T$, $T \to T * F \mid F$, $F \to$ `num` $\mid (E)$. Derivação: $E \Rightarrow T \Rightarrow T * F \Rightarrow F * F \Rightarrow (E) * F \Rightarrow \cdots \Rightarrow$ `(1+2)*3`.



Árvore concreta (17 nós) e AST (5 nós). As cadeias $E \to T \to F$ só impõem a precedência e os parêntesis só agrupam. A AST tem **um nó por operação** (`Mult`, `Add`) e um por constante; o agrupamento `(1+2)` está na **forma** da árvore (o `Add` é filho do `Mult`).

Exemplo resolvido **Comandos: if-then-else**

$Stm \to$ `if` Exp `then` Stm `else` $Stm \mid \ldots$ Na árvore concreta **cada palavra reservada é um nó**; na AST `if/then/else` é **um só nó** com três filhos:



`then` e `else` só servem ao analisador para saber onde acaba cada parte; depois, a posição do filho já diz isso.

Vantagens da sintaxe abstrata:

- **simplifica as fases seguintes**, que não lidam com os detalhes da gramática;
- a análise semântica e a geração de código ficam **menos ligadas à linguagem-fonte**;
- facilita o **açúcar sintático**: o analisador reescreve logo $x > y \longrightarrow y < x$ e $x >= y \longrightarrow y <= x$, e as fases seguintes só implementam **2 comparações em vez de 4**.

Representar sintaxe abstrata

O compilador tem de **construir e processar** ASTs. O exemplo é a linguagem de **programas sequenciais** (Folha lab. 3, Exercício 2): instruções `ident = exp`, `ident++` e `stm1; stm2`; expressões com identificadores, números e operadores binários.

Linguagens funcionais (Haskell, ML, OCaml, F#)

Definição **AST como tipo algébrico**

Cada **categoria sintática** (instrução, expressão, operador) é um **tipo algébrico**: alternativas etiquetadas por **construtores**, cada um com **atributos** de tipos e número diferentes, possivelmente **recursivas**. Distinguem-se por **encaixe de padrões**.

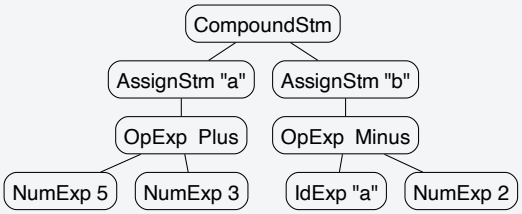
```haskell
data Stm = AssignStm String Exp     -- ident = exp
         | IncrStm String           -- ident++
         | CompoundStm Stm Stm      -- stm1; stm2

data Exp = IdExp String             -- x, y, z, etc.
         | NumExp Int               -- 123, etc.
         | OpExp Exp BinOp Exp      -- e1+e2, e1*e2, ...

data BinOp = Plus | Minus | Times | Div
```

Exemplo resolvido **A AST de a = 5+3; b = a-2**

```haskell
example :: Stm
example =
  CompoundStm
    (AssignStm "a"
       (OpExp (NumExp 5) Plus (NumExp 3)))
    (AssignStm "b"
       (OpExp (IdExp "a") Minus (NumExp 2)))
```



Processa-se com uma função por casos, um por construtor, com equações ou com `case`:

```haskell
process :: Stm -> ...
process (IncrStm id)         = ...
process (AssignStm id exp)   = ...
process (CompoundStm s1 s2)  = ...
```

```haskell
process stm = case stm of
   IncrStm id         -> ...
   AssignStm id exp   -> ...
   CompoundStm s1 s2  -> ...
```

Linguagem C

Definição **AST em C**

- Uma **`struct`** por categoria sintática, com uma **etiqueta** (*tag*, um `enum` com a alternativa) e uma **`union`** das alternativas; cada alternativa com mais de um atributo é ela própria uma `struct`.
- **Funções construtoras** que inicializam todos os campos.
- **Estilo funcional**: estruturas **imutáveis** e memória **libertada só no fim** (de uma fase ou do programa).
- Distinguem-se alternativas pela **tag** (num `switch`).

```
struct _stm {
  enum {COMPOUND, ASSIGN, INCR} tag;
  union {
    struct { struct _stm *fst, *snd; } compound;          // COMPOUND
    struct { char *ident; struct _exp *expr; } assign;    // ASSIGN
    char *ident;                                          // INCR
  };
};
typedef struct _stm *Stm;
```

```
typedef enum {PLUS, MINUS, TIMES, DIV} binop;
struct _exp {
  enum {ID, NUM, OP} tag;
  union {
    int val;                                              // NUM
    char *id;                                             // ID
    struct { binop op; struct _exp *left, *right; } binop;// OP
  };
};
typedef struct _exp *Exp;
```

Os construtores reservam memória e preenchem a *tag* e os campos da alternativa:

```
Exp mk_num(int v) {
  Exp e = (Exp) malloc(sizeof(struct _exp));
  e->tag = NUM;
  e->val = v;
  return e;
}

Exp mk_ident(char *txt) {
  Exp e = (Exp) malloc(sizeof(struct _exp));
  char *str = malloc(strlen(txt)+1);
  strcpy(str, txt);     // "clonar" a cadeia
  e->tag = ID;
  e->id = str;
  return e;
}
```

```
Exp mk_op(binop op, Exp e1, Exp e2) {
  Exp e = (Exp) malloc(sizeof(struct _exp));
  e->tag = OP;
  e->binop.op = op;
  e->binop.left = e1;
  e->binop.right = e2;
  return e;
}
```

(E o mesmo para `mk_assign`, `mk_incr`, `mk_compound`.) A cadeia clona-se porque, numa ação do Bison, o `txt` vem do `yytext` do Flex, que o próximo *token* reescreve †.

```
Stm example =
    mk_compound
    (mk_assign("a", mk_op(PLUS, mk_num(5), mk_num(3))),
     mk_assign("b", mk_op(TIMES, mk_ident("a"), mk_num(2))));
```

Nas ações do Bison:

```
exp : term             { $$ = $1; }
    | exp '+' term     { $$ = mk_op(PLUS, $1, $3); }
    ;
term : TOK_NUM         { $$ = mk_num($1); }
     | '(' exp ')'     { $$ = $2; }
     ;
```

Aqui os valores **não têm todos o mesmo tipo** (`TOK_NUM` é `int`, `exp`/`term` são `Exp`), por isso usa-se `%union` em vez de `%define api.value.type` †:

```
%union { int num; Exp exp; }
%token <num> TOK_NUM
%type  <exp> exp term
```

e no Flex escreve-se no campo certo: `yylval.num = atoi(yytext);`.

Análise de casos com `switch` sobre a *tag* (cada caso só deve usar os campos da sua alternativa, porque o C não o impede):

```
void process(Stm stm) {
  switch(stm->tag) {
  case COMPOUND: /* stm->compound.fst e stm->compound.snd */ break;
  case ASSIGN:   /* stm->assign.ident e stm->assign.expr  */ break;
  case INCR:     /* stm->ident                            */ break;
  }
}
```

Java e linguagens OO

> **Definição** **AST em Java**
>
> Uma **classe abstrata** por categoria sintática e uma **subclasse** por alternativa, com um **construtor** que inicializa todos os atributos; objetos **imutáveis**. Distinguem-se com **`instanceof`** e ***downcasts*** (ou com o padrão ***visitor***).

```
public abstract class Stm {}
public class CompoundStm extends Stm {
  public Stm fst, snd;
  public CompoundStm(Stm s1, Stm s2) { fst = s1; snd = s2; }
}
public class AssignStm extends Stm {
  public String id;
  public Exp exp;
  public AssignStm(String i, Exp e) { id = i; exp = e; }
}
```

```
public abstract class Exp {}
public class IdExp extends Exp {
  public String id;
  public IdExp(String i) { id = i; }
}
public class NumExp extends Exp {
  public int num;
  public NumExp(int n) { num = n; }
}
public class OpExp extends Exp {
  public Exp left, right;
  public int oper;
  final public static int Plus=1, Minus=2, Times=3, Div=4;
  public OpExp(Exp e1, int op, Exp e2) { left = e1; oper = op; right = e2; }
}
```

Construção e análise de casos:

```
Stm prog =
  new CompoundStm(new AssignStm("a",
                    new OpExp(new NumExp(5), OpExp.Plus, new NumExp(3))),
                  new AssignStm("b",
                    new OpExp(new IdExp("a"), OpExp.Times, new NumExp(2))));

public void process(Stm stm) {
  if (stm instanceof CompoundStm) {
    CompoundStm cstm = (CompoundStm) stm;  ...
  } else if (stm instanceof AssignStm) {
    AssignStm astm = (AssignStm) stm;  ...
  } else ...
}
```

Outras linguagens

Scala (*case classes*), Kotlin (*data classes*), Swift e Rust (*enumerations*) têm tipos algébricos e encaixe de padrões. **Para ASTs, prefere-se a abordagem funcional à OO**: em Haskell cada caso recebe os campos com nome e o compilador avisa se faltar um caso; em C é preciso disciplina com a `union`; em Java fica uma cadeia de `instanceof` e *casts*.

Pratica agora **Construir e processar ASTs (exercícios inventados)**

Ex. C Escreve a AST do programa `x = 1; y = x * (2 + 3); x++`, agrupando as instruções à direita (`x = 1; (y = ...; x++)`):

- **(a)** como valor Haskell, com os tipos `Stm`/`Exp`/`BinOp` acima;
- **(b)** em C, com os construtores `mk_...` (escreve também `mk_assign`, `mk_incr` e `mk_compound`, que os slides não mostram).

Ex. D Escreve um **interpretador** em Haskell para a linguagem dos programas sequenciais: `exec :: Env -> Stm -> Env`, com `type Env = [(String, Int)]` (o valor atual de cada variável). Usa encaixe de padrões, como no `process` acima, e uma função auxiliar `evalExp :: Env -> Exp -> Int`. Testa com o programa do Ex. C: no fim deve dar `x = 2` e `y = 5`.

Soluções: Ex. C (a), Ex. C (b), Ex. D